

Exploring Altered Open Source Software Development Patterns in a Time of Generative AI

Xinyu Li^{1*}, Francis Joseph Costello^{1*}, Keongtae Kim¹, and Xiaoquan (Michael) Zhang¹

¹The Chinese University of Hong Kong, CUHK Business School

xinyu.li@link.cuhk.edu.hk, francisjosephcostello@cuhk.edu.hk, keongkim@cuhk.edu.hk, zhang@cuhk.edu.hk

January 12, 2026

Abstract

Recent advances in generative artificial intelligence (AI) have enabled tools that can automate elements of knowledge work, with open source software (OSS) development seeing an influx of AI-based assistants. While these tools show promise in increasing productivity, little is known about how developers adapt to and alter their work patterns in response to novel coding assistants. We leverage the March 2023 announcement and April 2023 public release of GitHub Copilot X (CX) as a natural experiment, analyzing a 27-week user-week panel of 1,350 GitHub developers using difference-in-differences with coarsened exact matching. We further augment our GitHub data with matched LinkedIn career trajectories over the subsequent 6–12 months. We find that treated developers increase commit frequency by approximately 12% and active repositories by 6%. Beyond volume, work patterns shift toward broader engagement, with casual developers driving a 48% increase in work hours and increasing their exploration of new programming languages, while active developers show no significant increase in time input. However, this expansion comes with trade-offs in code quality, as we document a 2 percentage-point increase in copy-pasted code and a decline in project novelty for casual users. Finally, we find that AI-enabled shifts in development behavior are associated with a higher likelihood of internal promotions in the short- to medium-term. Overall, our results indicate that advanced AI coding assistants reduce participation frictions and enable altered working patterns, especially benefiting less established developers.

Keywords: open-source software, generative AI, future of work

* Authors contributed equally.

1 Introduction

Contemporary open source software (OSS) has come a long way since its early development days, now encompassing advanced technologies in diverse domains, such as artificial intelligence (AI), quantum computing, big data, and analytics (Hoffmann et al., 2024b). OSS communities have had to endure a number of exogenous shocks: from a generational pandemic (Malgonde et al., 2020) to technological acquisitions shown to reallocate contributor efforts and, in turn, alter project dynamics (Chen et al., 2022). As the survival ethos of the project is based on the contributions of developers to the project, the existing literature has shown that in a time of exogenous shock, OSS developers adapt well, either due to digital resilience (Malgonde et al., 2020) or because there is a project overlap between the target and acquiring firms (Chen et al., 2022). However, in 2021, the introduction of GitHub Copilot posed a novel and unprecedented exogenous shock—one that, at its core, redefines the relationship between human developers and their work. Although previous shocks tested the resilience and adaptability of the community, the emergence of GitHub Copilot introduced a new disruption with far-reaching implications for high-skilled workers typically embedded in OSS communities (Cui et al., 2024; Felten et al., 2019).

GitHub Copilot, an AI tool capable of writing, explaining, and generating test code, has emerged as a leading AI coding assistant (ACA). Originally launched as a technical preview in 2021 (Xu et al., 2025), it evolved from basic autocompletion into a "cornerstone" technology (Harding & Kloster, 2024) with the potential to boost global GDP by over \$1.5 trillion (Dohmke et al., 2023). While initial releases were welcomed, Copilot's profound impact on OSS communities crystallized with the March 2023 announcement of GitHub Copilot X (CX). Powered by OpenAI's latest generation of GPT-based models, CX promised to transform coding through advanced code generation and project management. Upon release, GitHub reported swift adoption: over one million developers and 20,000 organizations in its first year, effectively equivalent to adding 15 million developers with AI (Dohmke et al., 2023). Thus, CX equipped developers of all levels with powerful tools for personal and collaborative projects (Sarkar et al., 2022).

Market insights show that ACAs have begun to play an increasingly prominent role in the toolkit of OSS developers (Cui et al., 2024; Dohmke et al., 2023). Recent literature analyzing Copilot's initial release reveals two distinct features of ACA adoption in OSS communities. The

first stream demonstrates productivity gains of up to 58% (Hoffmann et al., 2024a; Peng et al., 2023) and code quality improvements showing enhanced maintenance contributions among high-quality programmers (Yeverechyahu et al., 2024). Contrasting evidence showed increased productivity and code quality primarily for less experienced users (Kreitmeir & Raschky, 2024), aligning with findings that novices rely more on generative AI (Li et al., 2024; Li & Kim, 2025). The second stream focuses on organizational implications, revealing important trade-offs in ACA adoption. Findings suggested that while Copilot increased successful code contributions by 6.5% and improved individual productivity, it also increased integration time by 41.6% due to higher coordination costs. Findings also revealed that usage was higher among core developers meaning they were more likely to obtain the benefits than peripheral developers (Song et al., 2023).

Emerging evidence has shown that generative AI is having a profound impact on the coding community (Li & Kim, 2025), with early versions of generative AI use, including GitHub Copilot, demonstrating sizable productivity gains in development processes (Cui et al., 2024; Dohmke et al., 2023; Hoffmann et al., 2024a; Peng et al., 2023; Song et al., 2023; Yeverechyahu et al., 2024). Yet, it remains unclear whether these effects generalize when coding barriers are further reduced through more advanced, conversational interfaces that support natural-language interaction with ACAs. CX introduces this change, allowing developers to seek and obtain programming information through queries of the codebases in natural language (see Figure A2 for a visual of the chat functionality in the IDE), sift through large volumes of text and code, and obtain concise summaries of lengthy documentation for libraries and application programming interfaces (APIs) (Nam et al., 2024). Given this innovation in how developers can approach their development activities, we focus on this major evolution of Copilot for several reasons: (1) it allowed for a roughly eightfold increase in context window and model size relative to earlier versions (a substantial leap in large language models (LLMs) since the initial release of ChatGPT); (2) it substantially improved the system’s ability to represent rich code context and generate coherent multi-line code blocks (OpenAI, 2024); (3) due to enhanced memory, the output was less likely to be hallucinated by the LLM, thus providing more accurate responses. As a result, CX effectively transforms Copilot from a simple autocompletion utility into a full-fledged AI coding assistant (Dohmke et al., 2023). Furthermore, this wave of technological improvements coincides with a sharp rise in public awareness and adoption of generative AI for coding, as reflected in the proliferation of dedicated coding assistants such as

Replit and Codeium (Codeium, 2023; Replit, 2023). Taken together, these developments motivate our decision to study GitHub CX by exploiting its March 22, 2023 announcement and April 2023 public release as discrete shocks for empirical identification.

Building on research into AI and OSS development, we argue that advanced ACAs can reshape developers’ work patterns by lowering the knowledge and coordination frictions that traditionally constrain sustained participation in OSS projects (AlMarzouq et al., 2015, 2023; von Krogh & von Hippel, 2006). Consistent with emerging evidence that Generative AI reallocates effort across tasks for knowledge workers (Brynjolfsson et al., 2025; Dillon et al., 2025; Eloundou et al., 2024), CX may allow OSS developers to offload routine search and comprehension tasks, reduce the burden of navigating unfamiliar architectures and dependencies, and engage more broadly across repositories and technical domains. Yet existing ACA studies in OSS focus predominantly on short-run, task-level productivity and have yet to clarify whether conversational, context-rich assistants enable deeper and broader contribution patterns, alter quality outcomes, or reshape the career-relevant signals embedded in developers’ GitHub footprints (Daniel et al., 2025; Hann et al., 2013; Nagle, 2018). Building on the established productivity gains documented in research on Copilot, we address three fundamental questions: (1) Do productivity gains observed in the initial release of Copilot extend to CX? (2) How does the use of ACAs alter OSS developers’ work patterns? (3) Does ACA-supported work exhibit higher quality outcomes?

CX provides a novel setting in which to answer these RQs and thus we analyze a panel of developers, using Pro user status as a proxy for CX usage in our treatment group. Using an intent-to-treat difference-in-differences (DID) empirical setting around CX’s public release, we find treated developers exhibit productivity gains in both commits ($\approx 12\%$ increase) and active repositories ($\approx 6\%$ increase) compared to the control group, addressing RQ1 and aligning with prior findings (i.e., Peng et al. (2023), Song et al. (2023), and Yeverechyahu et al. (2024)). Regarding work patterns (RQ2), we observe that developers do not simply code more, but code differently. In addition to the commit volume increases, the substance and nature of these contributions alters significantly, evidenced by a 63% increase in meaningful line changes per commit and longer intermissions between sessions, consistent with a reduction in costs incurred for context switching enabled by ACAs. Crucially, heterogeneity analysis reveals that ACAs affect developers differently depending on experience. The observed increase in work hours is driven entirely by casual

developers ($\approx 48\%$), whereas highly active developers show no significant increase in time spent. This extends to exploration activities too, where we find a 68% increase in new programming language adoption, an effect concentrated among casual users. Regarding quality (RQ3), we find mixed results: while we observe no significant change in short-term code churn, we document a 2 percentage-point increase in copy-pasted code, suggesting potential medium-to-long-term maintainability concerns. Additionally, project novelty declines among casual users, while active developers show no significant differences. Finally, we conducted a supplementary analysis utilizing matched LinkedIn data to examine whether these altered behaviors translate into tangible career outcomes. Our results suggest that treated developers are more likely to experience internal transitions and promotions in the short- to medium-term.

Our study contributes to ongoing discussions on how generative AI reshapes development patterns in OSS and, more broadly, in knowledge-based work (see Table A1 for a breakdown). First, we show that CX enhances productivity gains seen in the initial release of Copilot, especially on existing workflows, but also reorganizes development patterns. Relative to the control group, CX users increase their commit frequency and sustain contribution activity over longer horizons, while also shifting toward more exploratory development across programmatic files. At the same time, we document trade-offs in code outcomes, with gains in productivity and exploration accompanied by increases in copy-pasted code and reduced project novelty among casual users. Together, these findings move beyond traditional cost-based accounts of OSS participation by demonstrating how generative AI assistance reshapes the intensity, scope, and character of development patterns in OSS.

Second, we contribute to literature on generative AI at work (Hoffmann et al., 2024a) and open innovation by showing how different types of OSS developers use AI to manage the practical costs of contributing and, crucially, how they alter their working patterns. While Song et al. (2023) and Hoffmann et al. (2024a) found core developers benefited from initial ACA versions, our findings on CX show casual developers experiencing greater gains, demonstrated by both more meaningful contributions and increased programming language exploration. This aligns with Kreitmeir and Raschky (2024), suggesting AI tools particularly reduces the costs associated with switching contexts for less experienced developers. We identify three key mechanisms: increased code modifications, distributed workflow patterns with longer intermissions between sessions, and increased

technological exploration among casual developers. These findings contribute to our understanding of AI assistance (Baird & Maruping, 2021) by showing casual developers’ greater technical engagement in OSS projects with AI tools, suggesting an alternative pathway for less experienced developers to contribute to innovation in OSS development.

Third, we advance understanding of how AI reshapes the career-related benefits of OSS participation. Prior work shows that OSS contributions can support career advancement and respond to financial incentives (Conti et al., 2023; Hann et al., 2013), yet developers still face difficult trade-offs in allocating scarce time between OSS engagement and other career goals (AlMarzouq et al., 2023; von Hippel & von Krogh, 2003; von Krogh & von Hippel, 2006). Linking GitHub activity to LinkedIn career trajectories, we find that ACA access is associated with increased internal mobility—raising the likelihood of within-firm transitions and promotions in the first 6–12 months following the release of CX. This complements evidence that making developers’ technical skills visible through digital platform features (e.g., on GitHub) is associated with increased job promotions (Daniel et al., 2025), and extends Lerner and Tirole (2002) by showing how AI-enabled shifts in work patterns can accelerate, rather than delay, career returns to OSS participation. Overall, our results suggest that AI tools help developers alter development patterns allowing for more strategic contributions, enabling medium-term professional gains from OSS development while balancing day-to-day job demands (von Krogh et al., 2012). However, our findings also suggest a need for caution: while AI tools enable more strategic contributions and medium-term professional gains, the observed increase in copy-pasted code and decline in project novelty among casual users may indicate skill atrophy or over-reliance on AI assistance. This raises questions about whether employees—or the broader OSS ecosystem—will need to implement additional vetting mechanisms to ensure that the skills demonstrated through AI-assisted contributions remain robust and innovative over time.

2 Related Literature

We draw on the economic and organizational literature on OSS to understand the incentives and constraints shaping developers’ contribution decisions and development patterns. Early work conceptualizes OSS participation as a form of “private–collective” innovation in which individual de-

velopers invest private effort while simultaneously creating public goods (von Hippel & von Krogh, 2003). Within this perspective, contributions are motivated by a combination of delayed career benefits, reputation gains, and intrinsic enjoyment (Lerner & Tirole, 2002; Shah, 2006). Subsequent research develops this view into a broader economic and governance framework, emphasizing how project-level coordination mechanisms, licenses, and institutional arrangements shape participation and sustain collective production (Lerner & Tirole, 2005). Complementing these economic accounts, community-oriented studies highlight how knowledge sharing, mentoring, and socialization practices embed individual motivations in everyday project life (von Krogh et al., 2012). Related work shows that reciprocity and social exchange play a central role in sustaining contributions over time (David & Shapiro, 2008; Teng et al., 2024), and that developers actively balance learning, signaling, and community-building goals when engaging in OSS projects (von Hippel & von Krogh, 2003; von Krogh & von Hippel, 2006).

A second strand of OSS research emphasizes the frictions that make sustained participation and contribution difficult, particularly in complex, collaborative projects. One of the key challenges, especially for less experienced or peripheral developers, lies in overcoming substantial knowledge barriers such as incomplete understanding of the architecture, opaque dependency structures, and limited awareness of project norms that increase the effort required to contribute meaningfully (von Krogh & von Hippel, 2006). These burdens grow with project complexity and are especially pronounced for non-routine changes, which demand deeper comprehension of inter-dependencies and potential side effects. In advanced collaborative settings, developers must also coordinate with globally distributed contributors and navigate overlapping activities without formal task allocation (AlMarzouq et al., 2015). From a cognitive perspective, developers have finite mental resources that must be dynamically allocated across reading documentation, understanding existing code, coordinating with others, and writing new code (Kurzban et al., 2013; Maruping & Daniel, 2019). The flexible temporal scheduling characteristic of OSS development enables developers to adaptively allocate effort within individual projects or across project portfolios, presenting a canonical task-juggling problem commonly conceptualized as multitasking (Coviello et al., 2014). When activity levels remain moderate, multitasking often functions as a productivity enhancement mechanism (Kurzban et al., 2013), with information technology facilitating asynchronous coordination that increases aggregate output (Aral et al., 2012). However, as activity intensity escalates, cognitive

switching costs begin consuming proportionally larger temporal fractions than productive work execution, thereby attenuating productivity benefits (Coviello et al., 2014, 2015).

Recent research has begun to examine how AI coding assistants influence OSS development, with studies of early GitHub Copilot versions documenting improvements in task-level productivity and shifts in how developers generate and assemble code (Cui et al., 2024; Dohmke et al., 2023; Hoffmann et al., 2024a; Li & Kim, 2025; Peng et al., 2023; Song et al., 2023; Yeverechyahu et al., 2024). However, this emerging evidence largely reflects autocomplete-style assistance and short-run outcomes, leaving open questions about whether more advanced, conversational systems reshape the deeper frictions highlighted in the OSS literature. CX extends prior tools by enabling natural-language interaction, summarization, and context-rich code understanding (Nam et al., 2024), potentially lowering the cognitive burden of exploring unfamiliar repositories and coordinating contributions across modular, interdependent codebases, thereby tightening the conceptual link between ACA adoption and classic OSS barriers around architecture comprehension, dependency navigation, and norm internalization (AlMarzouq et al., 2015, 2023; von Krogh & von Hippel, 2006). Broader evidence on generative AI shows that LLM tools can deliver sizable productivity gains, often with particularly large benefits for less-experienced workers (Brynjolfsson et al., 2025; Hoffmann et al., 2024a; Noy & Zhang, 2023). Rather than replacing whole jobs, these tools typically automate specific subtasks and change how workers allocate their own time and attention (Dillon et al., 2025), with macro frameworks showing that AI adoption proceeds primarily through task reallocation within jobs rather than sudden, full displacement (Acemoglu, 2025). Only a subset of tasks are currently attractive to automate, implying that most workers experience task portfolio reconfiguration rather than full automation (Eloundou et al., 2024; Svanberg et al., 2024). A key mechanism enabling this reallocation is routinizing and modularizing work—by establishing clear interfaces between circumscribed human tasks and automated workflows, which reduces coordination costs and makes individual tasks more amenable to automation (Tschang & Almirall, 2021).

In practice, individuals tend to offload routine, codifiable work to generative AI, while spending relatively more time on higher-value tasks. For OSS developers, this perspective raises questions about how ACA access reshapes development patterns: which tasks they still perform themselves, how they balance learning versus visible contributions, and whether generative AI narrows or widens

gaps between developer experience levels. We address these questions by tracing how CX access is associated with changes in developers’ observed behavior post-release.

3 Empirical context and data

3.1 GitHub and GitHub Copilot

GitHub serves as a fundamental platform for OSS development, facilitating collaboration through activities such as committing changes to local repositories and pushing these changes to remote repositories, as illustrated in Figure 1. Commits are recorded changes to a repository created in the local computing environment. They reflect a temporary save of progress and can represent both incremental adjustments to a project and experimentation with code structure. Importantly, commits can be revised or discarded prior to publication, allowing developers flexibility to make mistakes and iteratively refine ideas. Once a developer reaches a stage where locally committed changes are ready to be shared with the OSS community—whether in personal or community projects—they can push these changes to a remote repository. Pushes therefore signal willingness to make work publicly visible and may reflect greater confidence in the quality or readiness of a contribution. AI coding assistants, therefore, operate within this workflow, affecting how developers write, test, and refine code before committing and pushing their work.

GitHub Copilot, designed to integrate directly into developers’ coding environments, has undergone several major evolutions since its initial release in 2021. CX, announced in March 2023, represented a significant technological leap from earlier versions. Whereas the original Copilot prototype, based on OpenAI’s Codex, focused primarily on inline code and comment completion, CX introduced multi-file context awareness and conversational code assistance powered by a more advanced GPT model. These capabilities expanded AI assistance throughout the development process, from initial code exploration to final refinement (see Appendix 1 for more details on programming characteristics and how Copilot and CX’s Chat tool affected coding characteristics).

We choose CX as the exogenous shock in our research design because its March 2023 announcement and April 2023 release represent a discrete, publicly observable event that affected GitHub developers differentially based on their subscription status. The key advance for our empirical strategy is that CX’s tighter grounding in locally available code and documentation through IDE

integration¹ should improve the coherence of suggestions and may reduce ungrounded or inconsistent outputs during complex development tasks. This creates a plausibly larger treatment effect compared to earlier autocomplete-style assistance, making CX an appropriate empirical setting to observe changes in software development patterns.

Finally, CX’s announcement-to-release sequence offers a natural two-phase setting for examining the impacts of introducing advanced generative AI features. CX was first announced on March 22, 2023, as an experimental feature collection reflecting GitHub’s vision for AI-empowered software development. The announcement was accompanied by a registration process that encouraged interested developers to sign up for a Copilot subscription and join a waiting list, with a limited technical preview granted to a subset of early adopters shortly thereafter. In late April, the chat functionality was released as a separate publically available extension on two supported IDE marketplaces, and users could install and activate the new features via these IDEs. A detailed timeline is shown in Figure 2. Together, these events—an initial announcement with restricted early access followed by broader availability of the chat extension—create distinct post periods that separate awareness and anticipation from direct exposure to CX’s conversational assistance. This structure enables comparisons of developers’ behavior across the pre-period and the two post phases. Moreover, the pre-release awareness phase coincided with a surge of public and developer interest in generative AI tools, as evidenced by Google search trend patterns for Copilot around both the initial launch period and the CX announcement phase (see Figure A3). For active OSS contributors, staying informed about emerging developer tools is inherently part of their professional practice. This combination—heightened visibility of AI coding assistants during a period of general interest and the professional imperative for experienced developers to monitor tool innovations (Rolandsson et al., 2011)—likely accelerated CX adoption among the most engaged developers.

3.2 Sample construction

We employed a snowball sampling strategy that began with the five most popular GitHub users and expanded through their starred-user networks, yielding a final sample of 4,021 developers. For each sampled developer, we extracted public event logs from GH Archive to identify their

¹Users can select code lines as context and send their inquiries to Copilot Chat in the sidebar to obtain code suggestions, explanations, unit tests, and potential fixes. See Figure A2.

commit activity. Using these commit IDs, we then obtained fine-grained commit metadata and code modification information through PyDriller (Spadini et al., 2018), which programmatically issues requests to GitHub’s REST API. Mirroring CX’s staged release, we created a user-week panel that spans 27 weeks: 12 weeks prior to the announcement, a 5-week pre-launch promotion/registration period, and 10 weeks post-launch. Users are retained if they contribute at least one commit in both the pre-announcement and any of two post-announcement windows.

One identification challenge in the sample construction stage is to identify the treated users, i.e., users who have access to CX. While the subscription status of Copilot is not publicly shown on users’ profiles, we can easily observe if a user has subscribed to the GitHub Pro plan, a premium service on GitHub. If subscribed, the Pro Label is automatically displayed next to his/her avatar. Here, we argue that users’ subscription statuses of this paid service can be considered a good proxy of treatment in our intention-to-treat (ITT) analyses, given that this premium service subscription is commonly bundled with a GitHub Copilot subscription. This strong binding relationship between the two services can be justified for two reasons. First, educational users can apply for access to these two paid services as a bundle for free through the GitHub Education scheme, where an educational developer pack covering these two services is provided to all verified educational users (both students and faculty).² Second, complementary purchases frequently occur in organizations’ software service procurement due to switching costs and established customer relationships (Chen & Hitt, 2006). Software providers’ cross-selling strategies and sequential marketing campaigns, particularly through automated email promotions to existing subscribers, further increase customers’ likelihood of adopting complementary products and services (Liu-Thompkins & Tam, 2013).

To verify this expected correlation between the Pro badge and likelihood of access to Copilot, we performed two ad-hoc robustness analyses with available data. First, we analyzed discussions about Copilot in GitHub’s official forum. By reviewing Copilot-related topics, we find that 436 out of 1,634 participants (26.7%) are Pro users, more than double the proportion found in GitHub’s active user population.³ This large magnitude difference suggests that Copilot use is much more prevalent among Pro users at the time of CXs’ launch. Next, we performed a further robustness check more specific to the sampled users. We trained a bespoke Deep Learning model to predict the likelihood

²<https://education.github.com/pack>

³As there are no public statistics on Pro user proportion, we estimate it using the same snow sampling method based on 762,000 users (approximately 38% of monthly active users according to open statistics).

of new code addition being either human or AI generated. Statistical testing verified that Pro (compared to non-Pro and within Pro status pre vs. post CX) users showed a significantly higher proclivity of having AI-generated code additions post CX (full details can be found in Appendix 7). Taken together, these two pieces of evidence support the validity of our ITT approach by showing that, in light of prior findings, holding a Pro subscription is a strong proxy for CX adoption and use.

Under the assumption that Pro plan subscribers are more likely Copilot users, we leverage users' pro status to construct treatment and control groups. Because the API exposes subscription status only at the time of data collection, we additionally verified timing using the Wayback Machine to confirm whether each user's Pro subscription began before the CX release. This review excluded 900 users who obtained Pro status immediately after our observation window and revealed a large cohort joining Pro in the first three months following the CX roll-out.⁴ Since developers self-selected into this premium plan, Pro users' characteristics may differ substantially from non-Pro users. To mitigate this self-selection bias, we construct comparable user pairs by matching on key characteristics and pre-treatment programming activities on GitHub. The matching variables encompass demographic characteristics (student and employment status), platform experience indicators (account tenure, number of repositories, proportion of forked repositories), and historical contribution patterns (measured by log-transformed counts of diverse contribution types in the three months before the announcement).

To fully leverage the data, we apply coarsened exact matching (CEM) using all observations, weighting control users by the ratio of treated to control units within each stratum. This procedure yields a matched sample of 1,373 users—567 treated and 806 controls—with stratum-specific weights carried forward into the subsequent DID analyses.⁵ After matching, standardized mean differences across covariates are generally below 0.2, indicating satisfactory balance between the treatment and control groups (see Figure 3). We further assess robustness using alternative matching approaches, including propensity score matching (PSM), and by expanding the matching set to incorporate users' historical development patterns (e.g., maintenance work ratio and work cadence), which we report in later sections.

⁴We also noticed a large batch of users joined the Pro plan in the first three months after the roll-out of CX.

⁵In our CEM implementation, each continuous variable is coarsened into 2–5 bins.

3.3 Measures

Our measurement approach centers on a foundational indicator of coding activity—public commits to users’ self-owned repositories—which, as previously discussed, constitutes one of the core building blocks of development on GitHub. Each commit provides granular documentation of file modifications, enabling precise tracking of specific lines added or removed within affected files. Leveraging the associated timestamp and committer metadata, we construct a user-level panel dataset with consecutive observations of weekly development activities, thereby generating a longitudinal representation of sustained engagement patterns and incremental project progression over time.

We first operationalize development intensity through two primary metrics: each user’s weekly commit frequency and the count of concurrently active projects (repositories). While the frequency of commits during a given temporal window provides a general proxy for development effort, a more granular analysis of modifications between consecutive commits allows for richer analytical insights into the nature and magnitude of developmental work. Accordingly, we compile a suite of fine-grained indicators that capture both workload intensity and code quality dimensions within weekly contributions. Within the open-source ecosystem, the precise code and file modifications encapsulated in each public commit are observable through the associated “diff” file (a structured data artifact that documents the sequence of line-level edits applied between successive file versions). Each modified file records discrete sequences of altered lines, which constitute units of editing activity and represent meaningful semantic changes to the codebase. Leveraging this granular diff metadata, we quantify both the scale and substantive character of developers’ weekly contributions, enabling an in-depth assessment of coding productivity and behavioral patterns.

Nevertheless, constructing valid and reliable measures of workload intensity and code quality in software development contexts presents substantial methodological challenges, attributable to considerable heterogeneity in coding conventions and marked variations in the complexity of developers’ contributions. Moreover, comprehensive analysis of granular diff data necessitates significant computational resources to effectively parse extensive modification files when executing raw code analytics at scale. Consequently, prior empirical investigations have predominantly employed coarse-grained proxies—including commit counts, pull request frequencies, or aggregate lines of

code—which inadequately capture the semantic depth, architectural sophistication, and long-term maintainability characteristics of developers’ contributions (Kalliamvakou et al., 2016). To address these measurement limitations, we adopt the metric framework articulated in GitClear’s technical documentation, which provides a theoretically-grounded and systematically operationalized approach to interpreting diff-level data.⁶ First, we implement a filtering mechanism that excludes non-programming file modifications and systematically removes auto-generated code patterns and repetitive line sequences, thereby constructing a *meaningful line changes* metric that isolates substantive developmental contributions.

Second, we classify each line-level operation (i.e., additions, deletions, modifications, and relocation) to characterize developers’ weekly editing behaviors. Building upon this classification, we follow GitClear’s proprietary indicators to help operationalize their metric *diff delta*, which quantifies coding effort magnitude through weighted scores calibrated to the estimated cognitive load of each operation type. Another measure of interest, *code churn*, was captured to show the proportion of code subsequently revised or removed within a specified temporal window, serving as an inverse proxy for code stability. Collectively, these measures provide a substantially more nuanced characterization of coding effort, output quality, and codebase maintainability relative to conventional aggregate metrics, thereby strengthening our analytical capacity to detect meaningful shifts in developer productivity within AI-augmented software development contexts. Beyond raw diff-based metrics capturing shifts in developers’ coding patterns, we incorporate unstructured data sources, including file extensions and textual metadata describing work content and project characteristics, to systematically examine ACA impacts on developers’ learning trajectories and maintenance behaviors.

We operationalize a user’s baseline programming breadth by enumerating the unique programming languages present across owned repositories, subsequently tracking the adoption of novel programming languages within submitted code contributions on a weekly basis. To capture project-level exploration behaviors, we construct a user-specific repository novelty measure whereby we extract the project’s textual data, including the README documentation for each newly created repository. We utilize Natural Language Processing to create semantic text embeddings so that we could compute the mean cosine distance between each new repository’s semantic embedding and the

⁶<https://www.gitclear.com/help>

centroid of embeddings derived from the user’s historical project portfolio (we utilized the preceding twelve-month period from the observational period). These text- and metadata-derived features complement line-level edit metrics to enable a multidimensional analytical framework for understanding exploratory behavior patterns within ACA-enabled development environments. Complete variable definitions and descriptive statistics are presented in Table 1.

Lastly, recognizing that users’ coding activities are contingent upon project lifecycle stage and accumulated development experience, we systematically collect comprehensive repository metadata—including repository name, creation timestamps, descriptive documentation, and associated web resources—alongside user profile attributes and longitudinal behavioral data. Our measurement construction process yields an analytical sample of 1,350 users for subsequent empirical analyses; of the original 1,373 matched users, 23 are excluded due to incomplete data coverage for either the focal user or their matched counterpart. Table 1 presents descriptive statistics for our primary analytical variables across these 1,350 users, encompassing 36,450 user-week observations in the panel structure. Upon inspecting the distributional properties, we find considerable heterogeneity in weekly development engagement: developers average 1.93 commits per week and actively contribute to 0.37 repositories. Temporal cadence and programming language portfolio expansion indicators demonstrate relatively sparse typical activity patterns (Sessions mean = 0.23) and modest weekly language adoption rates (Languages Added mean = 0.01). The substantial standard deviations across these metrics underscore pronounced variation in individual engagement trajectories, reflecting the inherently heterogeneous nature of OSS development activities and developer work patterns within the GitHub ecosystem.

3.4 Model specification

We estimate the intent-to-treat effects of ACAs on users’ development activities and work patterns through a two-way fixed effects difference-in-differences (DID) specification. Consistent with our identification strategy, we separate the post-CX period into two analytically distinct treatment phases. The first encompasses the post-announcement phase, during which enrollment occurred but the majority of users remained without functional CX access. For these users, they likely continued their reliance on the original Copilot implementation, with generative AI chat capabilities restricted to a limited early-access cohort. The second phase captures the post-launch period,

characterized by broad availability of CX’s advanced ACA chat functionalities across the treatment population. We operationalize these temporal phases through separate treatment-period interaction terms and incorporate CEM-derived propensity weights to enhance covariate balance between treatment and control groups. This empirical specification identifies treatment effects by comparing the differential evolution of development patterns between Pro subscribers (treatment group) and non-Pro users (control group) across the pre-treatment baseline period and the two subsequent post-treatment phases, thereby isolating the causal impact of CX access on observed behavioral outcomes. Formally, our empirical model is the following:

$$Y_{it} = \delta_1 \text{Pro}_i \times \text{PostAnnouncement}_t + \delta_2 \text{Pro}_i \times \text{PostLaunch}_t + \mathbf{X}'_{it}\beta + \alpha_i + \gamma_t + \varepsilon_{it}, \quad (1)$$

where Y_{it} denotes the observed value of dependent variables spanning users’ on-platform activity measures—including weekly commit and active repository counts—development pattern metrics, and code quality indicators for user i at week t . We apply logarithmic transformations to all count variables to address their pronounced right-skewed distributional properties. $\text{PostAnnouncement}_t$ is a time-varying indicator equaling one for weeks following GitHub’s public announcement of CX and its forthcoming AI-assisted development vision, but preceding the availability of enhanced Chat features through IDE marketplaces, while PostLaunch_t equals one during and after the official release of the chat-based ACA extension. Pro_i denotes user i ’s Pro subscription status at the announcement date.⁷ The vector, $\mathbf{X}'_{it}$, represents the set of time-variant control variables we discussed previously, encompassing the average development stage of repositories that user i actively contributed during the prior week, and encompassing the on-platform development experience, as defined in Table 1. We also incorporate user (α_i) and week fixed effects (γ_t) to account for time-invariant individual heterogeneity and common temporal shocks. ε_{it} represents the idiosyncratic error term.

⁷We extend the observation window for Pro status identification to two weeks post-announcement to accommodate typical adoption delays, as users frequently require time to become aware of and acquire access to major software updates.

4 Results

4.1 Effects of Copilot X on Overall Development Activity

We commence our empirical investigation by examining CX’s influence on the intensity of coding activities within the OSS development ecosystems. As illustrated in the OSS development workflow depicted in Figure 1, GitHub developers advance their projects through an iterative process comprising local code development and subsequent submission of modifications as commits to remote repositories. Accordingly, we first examine variations in commit frequency following treated users’ acquisition of CX access, benchmarked against the control group. The estimation results are presented in Table 2, with standard errors clustered at the individual user level to account for within-user correlation across repeated observations.

Table 2 presents our estimation results, commencing with a baseline specification incorporating solely user and temporal fixed effects before introducing controls for potential confounding factors. Column 1 reveals that the CX announcement is associated with a statistically significant increase ($\delta_1 = 0.1$, p-value < 0.01) in the weekly commit frequency among Pro users, relative to the control group. Following the public release of CX chat features on the IDE marketplaces, this positive treatment effect on commit volume demonstrates persistence, with the absolute marginal difference expanding from 0.16 to 0.21 commits per week.⁸ Upon incorporating controls for average project lifecycle stage and cumulative developer experience in column 2, the estimated treatment effects weaken modestly yet retain statistical significance ($\delta_1 = 0.09$, p-value < 0.05 ; $\delta_2 = 0.12$, p-value < 0.01). When compared against the sample mean baseline of 0.5 weekly commits, the coefficients in column 2 corresponds to a proportional increase of approximately 28% and 34% during the announcement and post-release phase, respectively. These empirical results corroborate prior investigations of the initial Copilot iteration, which documented elevated programming activity and accelerated OSS development trajectories following its public release (Song et al., 2023; Yeverechyahu et al., 2024).

While we observe elevated development activities among treated users, the underlying mechanisms driving these increases remain ambiguous. Specifically, it is unclear whether the additional commits stem from intensified development efforts within existing project portfolios or from

⁸The marginal commit change is calculated as $e^{\text{cons} + \delta_i} - e^{\text{cons}}$, where the constant equals 0.39 in column 1.

expanded project initiation behaviors. The consistent magnitude of coefficient estimates across columns 1-2 confirms an aggregate activity increase but does not disambiguate between these alternative explanations. To address this analytical gap, we examine granular evidence distinguishing repository creation activities from developmental contributions to pre-existing repositories. Figure 4 depicts the monthly temporal patterns of both repository creation and active repository contributions.⁹ The treatment shock in March 2023 precipitates an immediate surge in both newly created repositories and actively maintained repositories among treated developers. However, the increase in repository creation then exhibits a rapid mean reversal, returning to pre-treatment baseline levels within three months of the initial shock. In contrast, the increase in actively maintained repositories demonstrates substantial persistence over the extended observation period, even as the control group exhibits gradual convergence in late 2023. This divergent pattern—particularly the pronounced and sustained elevation in active repository contributions among treated users—provides evidence that ACAs primarily enhance development momentum rather than spurring new project initiations.

Building on the model-free evidence, we now formally analyze the sources of increased commits. To do so, we specify the count of repositories receiving commits each week as our dependent variable and regress it on our two DID interaction terms in column 3. The estimation results indicate that developers with elevated propensity for ACA access exhibit greater concurrent project engagement during the post-treatment period. On average, treated users contributed to 0.08 additional repositories per week relative to their outcomes absent CX access, representing a 35% proportional increase in active project portfolios compared to the control cohort.¹⁰ While modest in absolute terms, this relative increase is sizable against a baseline mean of 0.23 active repositories. Combined with the documented increase in weekly commit volume, it suggests that CX is associated with a higher baseline level of OSS engagement at both the project and commit level, rather than merely triggering one-off project initiations. In other words, developers appear more likely to remain active and to direct effort to a slightly broader set of repositories in a typical week.

⁹New project launches occur with insufficient frequency at the weekly granularity to yield meaningful statistical inference.

¹⁰For the baseline scenario, $\ln(\text{repos} + 1) = 0.23$ (constant term). Thus, the marginal effect equals $\exp(0.29) - \exp(0.23) = 0.08$ repositories.

4.2 Magnitude of Code Changes

While our findings reveal a rise in developers’ activity following CX introduction, we next investigate the compositional content of individual commits to distinguish whether this elevated activity reflects more substantive developmental work or simply heightened submission frequency driven by motivational factors. This examination is crucial because developers’ coding behavioral patterns may have fundamentally evolved in response to the availability of integrated, context-aware ACA capabilities. To empirically assess such granular shifts in users’ development processes, we collected detailed code modification metadata (“diff” files) from each commit submitted to public GitHub repositories from our sample population. We subsequently implemented comprehensive code analytics pipelines after filtering non-programming file modifications from our measurement construction. These commit-level measurements enable rigorous assessment of whether developers’ heightened platform engagement stems from genuine productivity gains manifest within individual commits, thereby providing analytical insights into how developers utilize AI assistance within their programming workflows.

Table 3 presents estimated treatment effects on average commit size as CX capabilities were progressively deployed across the user population. The analytical sample restricts to user-weeks containing at least one commit to ensure meaningful comparisons. The estimation results demonstrate that commits submitted by Pro users encompass more extensive code modifications following the full availability of CX features on IDE marketplaces (columns 1-3). Specifically, the incremental increase in line-level modifications within an average commit exhibits greater magnitude among Pro users relative to non-Pro users across both post-treatment temporal phases. While the announcement phase yields statistically insignificant treatment effects, the post-launch phase demonstrates a statistically significant effect whereby we observe a 32% proportional increase in line changes among Pro users, indicating that the operational deployment of CX capabilities corresponded to meaningful shifts in per-commit code contributions and intensity.

Beyond raw lines of code (LOC) changes, which systematically underestimate true cognitive coding effort (Shihab et al., 2013), meaningful line changes (column 2) exhibit stronger ITT effects that intensify substantially following the public deployment of CX’s enhanced capabilities to authorized users. A parallel pattern emerges for Diff Delta, an empirically validated developer

effort metric that assigns operation-specific weighted scores to classified edit operations captured within meaningful line changes.¹¹ The increased magnitude and heightened statistical significance of treatment effects for meaningful line changes and Diff Delta collectively indicate that CX primarily augments developers’ substantive coding activities, facilitating the production of more usable and semantically meaningful code modifications rather than superficial or low-signal edits such as whitespace formatting, documentation comments, or extensive string literal assignments.

In contrast, column 4 demonstrates no statistically significant treatment effect at the file-volume level. This null finding indicates that the observed expansion in commit magnitude stems predominantly from intensified modifications within affected files rather than from the expansion across additional file scopes. These findings collectively confirm that developers are producing substantively deeper code contributions per commit, thereby establishing that the documented increase in activity reflects genuine productivity gains. However, a critical ambiguity persists regarding the mechanisms underlying this observed output growth. Specifically, it remains unclear whether the increase stems from intensified effort allocation (whereby developers invest extended work hours to generate greater code volume), or from enhanced productivity efficiency (whereby developers generate equivalent or superior output within constant or reduced time intervals). We address this analytical question by examining developers’ work cadence patterns and temporal resource allocation in the subsequent section.

4.3 Effort Reallocation and Distributed Workflow

We analyze developers’ invested effort to identify the potential mechanisms underlying the output gains documented in preceding sections and to investigate potential effort redistribution dynamics within the development process. Recognizing that multiple commits are frequently aggregated and submitted to remote repositories as cohesive development tasks, we cluster each developer’s commits into discrete “work sessions” representing a continuous blocks of time allocated to development activities. A principal methodological advantage of measuring time investment through

¹¹Diff Delta represents a coding productivity metric developed by GitClear, a prominent commercial developer analytics platform provider (alongside Code Climate and LinearB). This metric quantifies “durable” code contributions by adjusting for measurement noise inherent in conventional metrics such as Lines of Code and Commit Count. GitClear’s validation studies demonstrate that Diff Delta exhibits stronger correlations with longitudinal code evolution patterns and actual developer effort expenditure than traditional aggregate measures. Comprehensive technical documentation regarding this metric’s construction and empirical superiority is available in their technical documentation and industry white papers (www.gitclear.com/help/technical/diff_delta_calculation).

work sessions rather than individual commits derives from the infeasibility of assigning uniform time allocations when only a precise initiation timestamp for each commit is observable. Instead, we specify a temporal interval for each cluster of functionally related commits by leveraging the timestamp pair associated with the initial and terminal commits within each session. Through systematic adjustments applied to interval boundaries and appropriate treatment of outliers,¹² we construct weekly working time estimates capturing each user’s aggregate effort investment across the observation period.

Table 4 presents estimated treatment effects on time investment for Pro users relative to non-Pro users. The estimation results reveal no statistically significant increase in users’ weekly time allocation following the full deployment of CX capabilities. A temporary elevation in working hours emerges during the announcement phase (column 1), yet this effect appears predominantly attributable to an increased frequency of session initiations (column 2) rather than extended session durations (column 3). This short-lived pattern aligns with brief exploratory behavior whereby developers experiment with newly announced capabilities amid broader enthusiasm surrounding generative AI tool proliferation, rather than reflecting systematic intensification of working hours. Upon looking at the post-launch phase, coefficient estimates across all three specifications indicate negligible magnitudes in time allocation changes, both at the aggregate weekly level and at the per-session granularity. Collectively, these empirical findings suggest that the documented output expansion cannot be attributed to heightened time investment into development activities. Instead, the evidence indicates that ACAs effectively reduce the marginal cost of code production while simultaneously enhancing per-unit productivity. This productivity shift enables developers to accomplish substantively greater work volumes within approximately equivalent temporal windows during active development weeks, suggesting efficient reallocation of cognitive resources toward more meaningful coding activities.

Having established that productivity gains materialize without corresponding increases in aggregate working time, we next examine whether CX adoption restructures the temporal organization of development activity, potentially through reduced task switching costs (Kurzban et al.,

¹²We derive raw session duration by incorporating two temporal buffers—an initial 15-minute pre-commit buffer and a subsequent 30-minute post-commit buffer—to the specified commit interval. Subsequently, we implement heuristic adjustment protocols including discounting isolated singleton commits and recalibrating anomalously high-frequency commit sequences that likely reflect automated processes rather than sustained human coding effort.

2013; Rubinstein et al., 2001). Table 5 reveals that despite exhibiting no statistically significant change in the absolute count of work sessions, the temporal architecture of development activity shifts markedly following CX deployment. Column 1 demonstrates that average intermission duration between consecutive sessions increases substantially. When considered alongside our prior finding that CX users actively contribute to significantly more repositories, these extended intermissions may reflect project-level multitasking behavior: developers distribute their constrained temporal budgets more sparsely across parallel project portfolios rather than concentrating effort within continuous blocks devoted to individual initiatives. Column 2 provides additional support, demonstrating elevated standard deviation in daily activity levels that indicates greater temporal fragmentation across the weekly cycle. These patterns align with emerging evidence that generative AI capabilities may attenuate cognitive friction in context switching (Brynjolfsson et al., 2025), as CX features including contextual code summarization and natural-language interaction effectively reduce the overhead of resuming dormant projects. Critically, this temporal restructuring operates predominantly within developers’ pre-existing schedules rather than expanding into discretionary leisure periods, as column 3 demonstrates no pronounced treatment effect on weekend contribution patterns among average Pro users. Nevertheless, subsequent heterogeneity analyses reveal that developers exhibiting historically sparse contribution patterns demonstrate substantively different behavioral responses, initiating more frequent sessions and increasingly utilizing weekend periods for expanded project engagement (see Tables 7 and A3).

4.4 Effects on Development Outcomes

We next examine whether the reduction in cognitive switching frictions extends beyond temporal reallocation to encompass expanded technical scope. Specifically, CX deployment may lower barriers to programming language adoption, enabling developers to engage with unfamiliar syntactic frameworks that previously remained inaccessible due to prohibitive learning costs. Traditionally, acquiring proficiency in novel programming languages imposed substantial cognitive investments in mastering syntax, idioms, and language-specific conventions—barriers that effectively constrained developers to working with familiar languages. This dynamic shifts fundamentally with natural language mediation capabilities embedded within CX, which enable developers to articulate algorithmic intent conversationally and receive syntactically correct implementations across diverse

programming environments. By abstracting away low-level syntactic details, these capabilities potentially democratize access to broader technical toolsets, allowing developers to experiment with and adopt languages based on functional requirements rather than pre-existing expertise constraints.

Column 1 of Table 6 presents evidence that CX deployment substantively increases programming language experimentation. Employing a Poisson specification appropriate for count data with low-frequency outcomes, we estimate that CX launch corresponds to a 68% increase ($e^{0.52} - 1$) in the rate of novel programming language adoption among Pro users relative to control developers. This finding aligns with the theoretical expectation that natural language mediation capabilities reduce the cognitive overhead traditionally required to produce syntactically valid code in unfamiliar linguistic environments. By enabling developers to leverage CX’s retrieval and code synthesis functionalities rather than relying exclusively on memorized syntax and language-specific conventions, these capabilities effectively lower barriers to technical diversification. Synthesizing this result with the documented temporal restructuring of development activity, the evidence indicates that CX primarily expands the exploration potential rather than development quality. In sum, developers maintain approximately constant temporal investments while distributing their fixed time budgets across wider project portfolios and more diverse technical environments.

While automation tools are often designed to improve human productivity in discrete one-time production processes (Acemoglu, 2025), software development is usually an iterated process coupled with maintenance and refactoring processes; it requires forward-looking planning to deliver durable software products. As highlighted in Google’s recent DORA reports, while ACAs improve process metrics such as code quality and readability for faster review and approval, they may compromise performance metrics such as delivery throughput and stability (Debellis et al., 2025). Although ACAs accelerate the production of code and help developers assemble larger commits, their assistance is anchored in the current visible state of the codebase. Unless developers explicitly include maintainability and forward-looking constraints in their prompts, AI-generated suggestions may favor near-term fixes or generic boilerplate code that do not fully anticipate future development trajectories, particularly in generation tools with limited customization for reaching the ultimate development goal.

At present, there is no consensus on how to map style-based text metrics like readability or

function length onto operational performance or maintainability, and because such metrics are highly endogenous to individual and team coding conventions, we rely instead on outcome-based measures of engineering quality. Specifically, we focus on the proportion of code rewritten shortly after submission (Code Churn %) and the proportion of duplicated, non-keyword lines (Copy-pasted %). These indicators directly reflect technical debt accumulation and maintenance burden while remaining predominantly agnostic to stylistic conventions across developers and programming environments.

We first assess short-run stability of development output using Code Churn % (Column 2, Table 6). As a proxy for near-term rework intensity, Code Churn % shows no statistically significant change in the post-announcement period, and the estimated effect becomes modestly positive after launch. These estimates suggest that CX does not immediately disrupt developers’ development processes, but neither does it materially dampen short-term rework volatility. We next examine medium-run structural effects. Column 3 indicates that CX adoption is associated with a persistent increase in code redundancy. The coefficient on Copy-pasted % is approximately 0.02 in both periods, implying an increase of about two percentage points in duplicated lines. This pattern is consistent with an AI-augmented workflow that relies more heavily on reusable templates and repeated implementation patterns. While such repetition can accelerate feature delivery, it may also introduce maintenance burdens by weakening abstraction and increase downstream refactoring costs (Haefliger et al., 2008). Finally, we test whether these efficiency-oriented shifts translate into greater innovative distinctiveness using Project Novelty (Column 4, Table 6). For each new repository, we assemble project text (including README documentation) and compute semantic similarity to the developer’s pre-observation portfolio; we define novelty as the inverse of this similarity. Column 4 shows no aggregate effect, indicating that expansion in activity does not automatically translate into more distinctive project themes. Heterogeneity results (Column 6, Table 7) further suggest a small decline in novelty among previously less active users, consistent with greater reliance on default AI suggestions when initiating new projects.

Taken together, the results in Table 6 reveal a tension between development exploration and quality within AI-augmented workflows. Along the exploration dimension, captured by technical scope expansion through novel programming language adoption (column 1), CX enables developers to distribute effort across broadened technical environments alongside the temporal fragmentation

patterns documented previously. Along the quality dimension, however, we observe concerning patterns. Technical quality indicators reveal statistically insignificant changes in code churn (column 2) and significant increases in code redundancy (column 3), suggesting that AI-augmented development introduces maintainability challenges through duplicated logic. Furthermore, project novelty remains unaffected by the intervention (column 4), indicating that CX does not enhance developers’ capacity to initiate genuinely innovative projects. These asymmetric effects suggest that while CX effectively reduces cognitive switching costs associated with context resumption and initiating work in unfamiliar technical domains, the tool does not substantively relax the architectural reasoning and creative ideation constraints that govern code maintainability and innovative output generation. Table 6 thus establishes empirically grounded boundary conditions for ACA-enabled productivity gains: within the observed temporal window, CX predominantly expands the exploration of development activity while simultaneously introducing technical debt through decreased code quality, with innovation outcomes remaining effectively unchanged by the intervention.

4.5 Additional Analyses

4.5.1 Heterogeneity: engaged v.s. casual developers

Existing studies suggest that the effects of generative AI tools vary systematically with users’ prior experience levels (Brynjolfsson et al., 2025). We conjecture that if ACAs significantly reduce the effort required for context familiarization and switching, they should increase developers’ marginal returns to handling more projects—especially for inexperienced developers, who often lack established practices and participate primarily for learning benefits (Hahn et al., 2008). Economic gains from OSS participation also differ by developers’ roles and activities (Hann et al., 2013), so identical cost reductions may impact contributor profiles differently. We therefore examine the heterogeneous effects of ACAs on GitHub users’ OSS participation and development patterns.

Specifically, we compare ACAs’ impact on casual developers and those with relatively stable pre-treatment development patterns, using average monthly commits in the three months before CX’s release to quantify historical participation intensity.¹³ Table 7 reveals distinct divergence in behavioral responses across these developer segments. For casual users, CX introduction serves

¹³We quantify users’ historical participation level by calculating their monthly average of commits submitted by each user in the three months before the shock.

as a catalyst for sustained participation increases. As shown in columns 1-4, these Pro users exhibit persistent and statistically significant elevations in development activity during both post-announcement and post-launch periods. The temporal patterns further reveal that during the post-announcement period, the majority of users experience increases in both activity levels and time investment. However, this broadly distributed increase gradually attenuates for Pro developers with stable development habits. This pattern suggests both heightened public interest following the announcement and a differential motivating effect of CX that disproportionately benefits the casual developer group.

Most notably, while the average treatment effect on working duration remained non-significant in the post-launch period (Table 4), casual users experienced substantial increases in time investment (47.2%)—both their session initiation volume (column 4 of Table 7) and session duration (column 1 of Table A3) rose significantly. This effect substantially diminishes and reverses for the most active users, who contributed similar volumes of meaningful code (column 2 of Table A2) while working fewer hours (column 3 of Table 7). Additional analysis in Appendix 5 confirms that distributed work patterns documented earlier primarily occurred among users with more casual users (Table A3). These asymmetric responses reveal that CX fundamentally alters participation dynamics for casual contributors. By reducing cognitive barriers to code comprehension and production, CX enables exploratory engagement patterns previously inaccessible to inexperienced developers. The documented increases in both session frequency and duration among these users suggest that ACAs effectively lower technical entry thresholds, transforming OSS development from an activity requiring sustained commitment into one accommodating intermittent, learning-oriented participation.

We also observe more pronounced effects on new language adoption among casual users (column 5), while CX appears to dampen language exploration among experienced users. This asymmetric pattern aligns with the effort-saving behavior documented earlier for experienced developers. In contrast, casual users exhibit declining project novelty (column 6). This pattern may appear inconsistent with existing evidence that inexperienced developers typically join projects requiring different technical expertise and derive higher learning benefits from diverse exposures (Hahn et al., 2008). However, because our study examines only users’ activities within their own repositories, casual developers likely continue pursuing external learning through collaborative participation

in others’ projects, where knowledge exchange and communication with experienced contributors remain valuable development pathways. The simultaneous decline in project novelty alongside expanded language adoption suggests that newly incorporated technical components serve primarily auxiliary functions rather than representing fundamental architectural shifts. While CX substantially enriches casual users’ language portfolios, it does not appear to catalyze genuinely disruptive innovation efforts beyond their established technical domains. Exploratory experimentation predominantly occurs at the boundary of existing capabilities, where ACAs’ inherent limitations in creative ideation constrain their capacity to overcome developers’ path-dependent trajectories.

4.5.2 Robustness tests

In this subsection, we examine whether our main conclusions remain stable under a broad set of alternative assumptions and empirical choices. We conduct a series of robustness tests that probe identification validity, reduce concerns about selection and specification, and assess sensitivity to alternative measures of key variables. Collectively, these checks provide convergent evidence that our results are not driven by any single modeling decision, sample construction choice, or measurement approach. (see Table 8 for a full overview).

Parallel trend assumption We test the parallel trends assumption by plotting dynamic treatment effects by period. Figure A4-A5 presents estimated coefficients for focal dependent variables in our main analyses. Overall, the estimated coefficients of the “leads” (i.e., weeks in the pre-treatment periods) are rarely statistically insignificant at the 5% level, indicating no significant differences between treatment and control groups across pretreatment weeks.

Alternative Matching Specifications To more stringently address potential self-selection bias, we implement a stricter CEM specification that augments our baseline pre-treatment covariates with additional measures of recent coding behavior—such as maintenance orientation and session-level activity—alongside developers’ country information to account for their demographic context. This richer coarsening scheme tightens the balance on coding intensity, persistence, and demographics, thereby further limiting unobserved heterogeneity relevant to Pro subscription decisions. Results based on this stricter CEM sample in Table A5 remain qualitatively consistent with and close in

magnitude to our main estimates.

We also employ PSM as an alternative to CEM to address concerns about our matching method selection. Specifically, we conduct one-to-one, without-replacement matching using the same set of covariates that predict both Pro plan subscription and our outcomes variables. This specification yields 529 matched pairs (1,058 users total). Analysis of this alternative matched sample corroborates our main findings, as shown in Table A6. Our results show significant increases in development activity at both the commit and repository levels. Although effect magnitudes are somewhat smaller than in our main analysis, the consistency in this alternative PSM specification supports the robustness of our primary conclusions.

Other Robustness Checks We implement various regression models to address potential model misspecification issues and employ diverse measurement proxies to mitigate measurement error concerns. Table 8 summarizes all robustness tests conducted.

4.6 Developer’s medium-term career prospects

While the preceding analyses demonstrate that CX access substantially increases development activities, a fundamental question remains: do these intensified activities translate into career advancement? Prior research establishes that sustained OSS participation yields long-term returns through contributors’ career advancement and enhanced labor market positioning (Lerner & Tirole, 2002), as development activities serve as credible signals of developers’ unobserved productivity characteristics (Hann et al., 2013). In this context, ACAs like CX may enhance career outcomes through two complementary mechanisms. First, by accelerating learning processes and expanding engagement, CX may improve performance and skill development. Second, as AI proficiency becomes an increasingly valued labor market credential,¹⁴ demonstrated facility with generative AI tools may signal technical adaptability while simultaneously catalyzing skill advancement.

To empirically examine these career implications, we augment our GitHub data with professional trajectories from LinkedIn, the dominant global platform for professional networking. We manually match users across platforms by systematically searching public LinkedIn profiles using identifying information from GitHub accounts, including username keywords and cross-platform social media

¹⁴<https://www.pwc.com/gx/en/news-room/press-releases/2024/pwc-2024-global-ai-jobs-barometer.html>

links.¹⁵ After rigorous identity verification using available biographical information, we successfully matched 2,978 users from our full sample to verified LinkedIn profiles. These comprehensive career histories extracted from LinkedIn resumes enable construction of job transition event sequences following Huang and Zhang (2016) and generation of covariates capturing pre-transition positions and accumulated work experience. Based on these constructed employment-related variables, we develop a cross-sectional analytical dataset comprising 470 developers from our matched sample.

Table 9 examines how ACA access affects career trajectories through Probit regression analysis of job transitions.¹⁶ We categorize transitions into external job hopping (employer changes) and internal transitions, encompassing both lateral transfers and hierarchical promotions. The estimation results reveal that ACA access significantly enhances career advancement, operating primarily through internal mobility channels. Pro users show higher probabilities of internal transitions (coefficient = 0.5, p-value < 0.05) in the first 6 months, with a consistently strong association (coefficient = 0.46, p-value < 0.01) extending to 12 months. Within these internal moves, we observe substantial promotion effects (coefficient = 0.46, p-value < 0.05) in the short term.

The longer-term analysis reveals persistent associations with internal mobility, though the promotion effect moderates somewhat (coefficient = 0.31, p-value < 0.1), potentially reflecting broader organizational adoption of AI development tools that diminishes relative advantages. Notably, we detect no statistically significant relationship with external job hopping, indicating that career benefits materialize predominantly through internal advancement pathways. This pattern exhibits theoretical coherence with our earlier empirical findings that AI-enabled productivity gains and technical experimentation concentrate primarily within personal rather than community repositories, rendering performance improvements more readily observable to current employers than to external labor markets. Moreover, because private organizational repositories remain unobservable in our data, the stronger internal mobility associations may partially reflect AI-enhanced contributions within firms' proprietary codebases that inform promotion decisions. While these results demonstrate that CX access correlates positively with enhanced internal career mobility, the precise causal mechanism (whether operating through enhanced signaling value of observable OSS

¹⁵We recruited master's students familiar with professional networking platforms to complete this matching task and excluded samples whose matched LinkedIn profiles exhibited low confidence levels.

¹⁶We implemented Logit and linear probability models as robustness checks. Both specifications returned qualitatively consistent results.

participation or through accelerated skill acquisition enabling superior task performance) remains an open empirical question requiring further investigation.

5 Discussions

This study examines how advanced AI coding assistants reshape working patterns in OSS development by reducing participation frictions. Using the staged announcement and public release of GitHub Copilot X (CX) as a natural experiment, we address three research questions: (1) whether productivity gains observed with earlier Copilot versions extend to CX’s more advanced conversational interface, (2) how ACAs alter OSS developers’ work patterns beyond simple productivity metrics, and (3) whether ACA-supported work exhibits different quality outcomes.

We find that access to CX is associated with higher development intensity and broader engagement. Treated developers increase commit frequency by approximately 12% and active repositories by about 6% relative to controls, while meaningful line changes per commit rise by roughly 63%. Intermissions between sessions also lengthen, consistent with lower context-switching costs and more flexible allocation of effort across repositories and files. These effects are highly heterogeneous: the increase in work hours (around 48%) and in new programming-language adoption is concentrated among previously casual developers, whereas highly active developers show no significant change in time spent. This pattern is consistent with a participation-expansion mechanism among casual contributors, complementing Kreitmeir and Raschky (2024) and contrasting with Song et al. (2023), who document greater usage among core developers for the initial Copilot release. One interpretation is that CX’s conversational interface and multi-file context reduce information-seeking and switching costs that previously constrained more casual users (Song et al., 2023; Yeverechyahu et al., 2024).

These participation gains have important boundary conditions for code quality and innovation. We observe an increase of roughly two percentage points in copy-pasted code, indicating greater reliance on duplicated or template-like solutions. Project novelty declines significantly for casual developers, while highly active developers show no significant change in novelty within our observation window. These patterns differ from Yeverechyahu et al., 2024, who document code-quality improvements among high-quality programmers using earlier Copilot versions. A plausible expla-

nation is compositional: whereas Yeverechyahu et al., 2024 study a setting in which adoption was concentrated among technically sophisticated users who could integrate inline completion into optimized workflows, CX’s conversational interface lowers adoption barriers and expands participation among less active developers who previously faced higher entry costs. Our quality findings therefore likely reflect shifts in the active developer pool rather than degradation among equivalent users. The increase in copy-pasted code and reduced novelty among casual contributors suggest that CX democratizes access faster than it cultivates the architectural judgment and design discipline developed through extensive practice. Overall, CX broadens participation and raises visible output for less active developers, while the cognitive and architectural demands of producing distinct, high-novelty innovations remain harder to support, generating an exploration–quality trade-off in ACA-supported development.

We also find that ACA access is associated with near-term career benefits. Linking GitHub activity to LinkedIn data, we observe higher rates of internal transitions and promotions within 6–12 months after CX’s release. This early evidence suggests a potential shift in the timing of OSS-related career returns. Distinct from the long-horizon external signaling emphasized in prior work (Hann et al., 2013; Lerner & Tirole, 2002), we posit that visible engagement with advanced AI tools may enable faster, within-firm rewards. Mediation analysis indicates that neither raw commit volume nor other observable development contribution changes account for these gains, consistent with the idea that, in labor markets where AI literacy is an emerging credential, visible AI tool adoption functions as a direct signal of adaptability and technical currency rather than merely a channel for higher observable output. Additional heterogeneity evidence suggests that this effect is even more pronounced for interns and junior employees, who often rely on such technical indicators to overcome a lack of professional history.¹⁷ This skill-signaling interpretation aligns with evidence that visibility of developers’ technical skills on GitHub is associated with promotions (Daniel et al., 2025), but we cannot observe employer decision processes or the specific attributes they reward. Accordingly, we treat signaling as a plausible—but not empirically verified—mechanism, and open questions remain about the durability of these benefits and their interaction with evolving organizational norms around AI-assisted contributions.

¹⁷Refer to Appendix 10 for the results of these complementary analyses.

5.1 Implications

Our findings have several implications for IS research and practice. First, they provide an alternative lens for examining costs in OSS development. While prior literature has primarily conceived of development costs as relatively stable constraints—whether in terms of knowledge acquisition (AlMarzouq et al., 2015, 2023), time and effort expenditure (Weber, 2005), or learning investments required to work across different technical domains—our findings suggest that generative AI solutions meaningfully alters this calculus. The introduction of context-aware chat-based ACAs reduces barriers that previously limited developers’ ability to initiate work, switch between projects, and explore unfamiliar languages. Evidence of this is reflected in our finding that developers in our treatment group exhibited significantly increased development activity across more diversified programming languages. This interpretation is further supported by our observed patterns of more distributed workflows with longer intermissions between sessions, particularly among casual developers who show substantial increases in work hours (48%) and session initiation. While prior work has treated time, effort, and learning investments as relatively fixed constraints that must be carefully managed (AlMarzouq et al., 2023; Weber, 2005), our findings suggest that AI can enable developers to reallocate these resources more flexibly across projects and technical domains, facilitating broader engagement patterns.

Second, our findings inform existing IS assumptions about expertise and technology adoption. Traditional models suggest that advanced tools primarily benefit expert users who can better leverage their capabilities (Song et al., 2023). However, our finding that developers with lower prior activity levels show an increase in programming language exploration, alongside an increase in hours spent on development, suggests a different pattern with advanced AI tools. This aligns with broader evidence from generative AI in knowledge work showing that LLM tools often deliver particularly large benefits for less-experienced workers (Brynjolfsson et al., 2025; Noy & Zhang, 2023), and extends these findings to the OSS context by demonstrating specific mechanisms—such as increased code modifications, technical scope expansion, and altered working patterns—through which less active developers leverage AI to overcome traditional barriers. These findings contribute to our understanding of technology adoption in software development by showing how AI’s capability to reduce search costs can alter traditional adoption barriers for more casual developers. The

contrast with Song et al. (2023), who found greater benefits for core developers with earlier Copilot versions, suggests that tool sophistication matters: conversational, context-aware interfaces appear to disproportionately benefit those who previously faced the highest barriers to participation.

Third, our career progression findings suggest a potential shift in the temporal realization of benefits from OSS participation (Lerner & Tirole, 2002, 2005). While early research showed that developers must endure a delayed period to realize career benefits, our LinkedIn data provides evidence that AI tools may be associated with accelerating this process. By enabling developers to contribute more substantially while managing their public contributions strategically, the use or signal of AI use may lead to faster realization of career benefits, as evidenced in the association with promotions observed in the 6–12 month period after the release of CX. This extends recent work on financial incentives in OSS (Conti et al., 2023) by showing how technological advances may be altering the temporal dynamics of OSS career outcomes (Baird et al., 2024). However, our mediation analysis indicates that raw commit volume does not explain these career gains, suggesting that the mechanism likely involves a more complex combination of factors—such as quality signals, demonstrated AI proficiency, increased visibility of technical exploration, or other aspects of contribution patterns—rather than any single measurable dimension of output.

Fourth, while our empirical setting centers on developers on Github, the mechanisms we identify are expected to extend to developers who use ACAs in organizational settings. Developers working on proprietary codebases often encounter comparable frictions as they navigate multiple projects and technologies and repeatedly reconstituting contextual understanding when resuming work. In some respects, our open source context may constitute a conservative case, to the extent that developers in our sample may have less access to formal onboarding processes and real-time collaborative support than those in corporate environments. However, we lack empirical means to validate these conjectures with our data, so our findings are best understood as evidence about open source development activity. We therefore view our results as providing indicative evidence into how ACAs may reconfigure work patterns in organizational settings, contingent on the institutional and governance differences that developers in such environments necessarily confront.

For practitioners, our findings suggest that AI coding assistance presents two distinct opportunities with important caveats. First, organizations with development teams may benefit from providing advanced AI tools to experienced developers, potentially enabling senior developers to

handle tasks traditionally assigned to junior developers through AI delegation, which could streamline operations. Conversely, these same tools appear to lower entry barriers for less experienced developers by democratizing access to complex development work, enabling them to tackle projects and explore programming languages that previously required substantial learning investment. This technological advancement has the potential to allow junior developers to be more experimental in their approach and learn through hands-on experience with complex projects. Organizations might consider pairing ACA access with mentoring or structured learning paths to ensure that breadth gains are complemented by depth development, though our results do not definitively prescribe specific program designs.

However, these opportunities come with significant quality and sustainability considerations. The increase in copy-pasted code and reduced novelty among casual contributors suggests that maintainers and organizations may need stronger review routines and tooling that prioritize contribution distinctiveness and maintainability alongside volume. These patterns align with recent evidence that AI-assisted programming may increase maintenance burdens over time (Xu et al., 2025), highlighting a critical limitation. While AI tools demonstrably help developers engage with more projects and increase output, they do not substitute for experienced developers in producing maintainable, innovative solutions. This asymmetry creates a potential organizational dilemma. Organizations facing short-term productivity gains from AI-augmented junior work may be tempted to adjust hiring strategies—retaining senior developers while reducing junior recruitment on the assumption that AI can substitute for entry-level contributions. However, such strategies risk disrupting traditional junior-to-senior progression paths through which developers historically acquired the architectural expertise and system design capabilities that remain difficult for AI to replicate. The eventual departure or retirement of senior developers from organizations that have curtailed junior hiring could compound the maintenance and technical debt challenges we document, leaving organizations with insufficient experienced personnel to manage increasingly complex codebases. Our career progression findings suggest that organizations should instead explicitly support AI tool adoption as part of structured developer career development programs that maintain pathways for juniors to develop the depth and architectural expertise that AI cannot yet replicate, even as short-term productivity gains from AI-augmented junior work appear favorable.

5.2 Limitations and Future Directions

Our study has several limitations that suggest important directions for future research. First, we focus on CX, a specific ACA whose capabilities and user experience differ from earlier versions such as the initial Copilot release; future work should compare initial releases and subsequent updates as these tools evolve. Second, our observation window captures only immediate and short-run responses, so we cannot examine whether casual developers who initially expand their breadth eventually deepen their engagement, whether observed quality trade-offs persist as users gain experience, or whether more experienced developers converge to similar patterns as they become accustomed to AI-generated suggestions; future longitudinal studies that track developers' behavior and code quality over longer horizons would be valuable to address these dynamics. Third, our empirical setting is limited to public GitHub repositories, and while we expect our mechanisms to be relevant beyond OSS, future work is needed to assess whether similar patterns arise in proprietary development environments with different incentive structures, oversight, and support resources. Fourth, our intent-to-treat design relies on Pro status as a proxy for CX access; although robustness checks support this choice, studies with more direct measures of generative AI usage could sharpen identification. Finally, while our LinkedIn data reveal suggestive associations with short-run career benefits, longer-horizon and business-cycle-spanning data are needed to understand how AI-assisted development shapes developers' careers over time.

References

- Acemoglu, D. (2025). The simple macroeconomics of ai. *Economic Policy*, 40(121), 13–58.
- AlMarzouq, M., Grover, V., Thatcher, J., & Klein, R. (2023). An empirical examination of newcomer contribution costs in established OSS communities: A knowledge-based perspective. *Internet Research*, 34(3), 665–689.
- AlMarzouq, M., Grover, V., & Thatcher, J. B. (2015). Taxing the development structure of open source communities: An information processing view. *Decision Support Systems*, 80, 27–41.
- Aral, S., Brynjolfsson, E., & Van Alstyne, M. (2012). Information, technology, and information worker productivity. *Information Systems Research*, 23(3-part-2), 849–867.
- Baird, A., & Maruping, L. M. (2021). The next generation of research on IS use: A theoretical framework of delegation to and from agentic IS artifacts. *MIS Quarterly*, 45(1), 315–341.

- Baird, M., Carpanelli, M., Xu, B., & Xu, K. (2024). Early evidence on the impact of GitHub copilot on labor market outcomes for software engineers. *LinkedIn Economic Graph Working Paper*, (5).
- Brynjolfsson, E., Li, D., & Raymond, L. (2025). Generative ai at work. *The Quarterly Journal of Economics*, 140(2), 889–942.
- Chen, P.-Y., & Hitt, L. M. (2006). Information technology and switching costs. *Handbook on economics and information systems*, 1, 437–470.
- Chen, W., Jin, F., & Xue, L. (2022). Flourish or perish? The impact of technological acquisitions on contributions to open-source software. *Information Systems Research*, 33(3), 867–886.
- Codeium. (2023). Codeium for enterprises [Accessed August 19, 2024]. <https://codeium.com/blog/codeium-for-enterprises>
- Conti, A., Gupta, V., Guzman, J., & Roche, M. P. (2023). Incentivizing innovation in open source: Evidence from the github sponsors program. *NBER Working Paper*, (w31668).
- Coviello, D., Ichino, A., & Persico, N. (2014). Time allocation and task juggling. *American Economic Review*, 104(2), 609–23.
- Coviello, D., Ichino, A., & Persico, N. (2015). The inefficiency of worker time use. *Journal of the European Economic Association*, 13(5), 906–947.
- Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2024). The productivity effects of generative AI: Evidence from a field experiment with GitHub copilot. *An MIT Exploration of Generative AI*.
- Daniel, S. L., Zhao, R., & Maruping, L. M. (2025). Digital skill visibility and job promotion of open source software developers. *Journal of Management Information Systems*, 42(3), 926–950.
- David, P. A., & Shapiro, J. S. (2008). Community-based production of open-source software: What do we know about the developers who participate? *Information Economics and Policy*, 20(4), 364–398.
- Debellis, D., Storer, K. M., Villalba, D., Harvey, N., D’Angelo, S., & Brown, A. (2025). *Impact of generative ai in software development* (Report No. v. 2025.2) (Licensed under CC BY-NC-SA 4.0). Google LLC, DORA (DevOps Research and Assessment). Mountain View, CA. <https://dora.dev/vc/genai/?v=2025.2>
- Dillon, E. W., Jaffe, S., Immorlica, N., & Stanton, C. T. (2025). *Shifting work patterns with generative ai* (tech. rep.). National Bureau of Economic Research.
- Dohmke, T., Iansiti, M., & Richards, G. (2023). Sea change in software development: Economic and productivity analysis of the AI-powered developer lifecycle. *arXiv preprint arXiv:2306.15033*.
- Eloundou, T., Manning, S., Mishkin, P., & Rock, D. (2024). Gpts are gpts: Labor market impact potential of llms. *Science*, 384(6702), 1306–1308.
- Felten, E., Raj, M., & Seamans, R. C. (2019). The effect of artificial intelligence on human labor: An ability-based approach. *Academy of Management Proceedings*, 2019(1), 15784.
- Haefliger, S., von Krogh, G., & Spaeth, S. (2008). Code reuse in open source software. *Management Science*, 54(1), 180–193.

- Hahn, J., Moon, J. Y., & Zhang, C. (2008). Emergence of new project teams from open source software developer networks: Impact of prior collaboration ties. *Information Systems Research*, 19(3), 369–391.
- Hann, I.-H., Roberts, J. A., & Slaughter, S. A. (2013). All are not equal: An examination of the economic returns to different forms of participation in open source software communities. *Information Systems Research*, 24(3), 520–538.
- Harding, W., & Kloster, M. (2024). Coding on copilot: 2023 data shows downward pressure on code quality. *GitClear*. Accessed: March, 2024, 1.
- Hoffmann, M., Boysel, S., Nagle, F., Peng, S., & Xu, K. (2024a). Generative AI and the nature of work. Available at SSRN 5007084.
- Hoffmann, M., Nagle, F., & Zhou, Y. (2024b). The value of open source software. *Harvard Business School Working Paper*, (No. 24-038).
- Huang, P., & Zhang, Z. (2016). Participation in open knowledge communities and job-hopping: Evidence from enterprise software. *MIS Quarterly*, 40(3), 785–806.
- Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., & Damian, D. (2016). An in-depth study of the promises and perils of mining github. *Empirical Software Engineering*, 21(5), 2035–2071.
- Kreitmeir, D., & Raschky, P. A. (2024). The heterogeneous productivity effects of generative ai. *arXiv preprint arXiv:2403.01964*.
- Kurzban, R., Duckworth, A., Kable, J. W., & Myers, J. (2013). An opportunity cost model of subjective effort and task performance. *Behavioral and Brain Sciences*, 36(6), 661–679.
- Lerner, J., & Tirole, J. (2002). Some simple economics of open source. *Journal of Industrial Economics*, 50(2), 197–234.
- Lerner, J., & Tirole, J. (2005). The economics of technology sharing: Open source and beyond. *Journal of Economic Perspectives*, 19(2), 99–120.
- Li, J., Mynatt, E., Mishra, V., & Bell, J. (2024). “always nice and confident, sometimes wrong”: Developer’s experiences engaging generative AI chatbots versus human-powered Q&A platforms. *arXiv preprint arXiv:2309.13684*.
- Li, X., & Kim, K. (2025). Impacts of generative ai on user contributions: Evidence from a coding Q&A platform. *Marketing Letters*, 36(3), 577–591.
- Liu-Thompkins, Y., & Tam, L. (2013). Not all repeat customers are the same: Designing effective cross-selling promotion on the basis of attitudinal loyalty and habit. *Journal of Marketing*, 77(5), 21–36.
- Malgonde, O., Zhang, H., Padmanabhan, B., & Limayem, M. (2020). Taming complexity in search matching: Two-sided recommender systems on digital platforms. *MIS Quarterly*, 44(1), 48–84.
- Maruping, L. M., & Daniel, S. L. (2019). Developer centrality and the impact of value congruence and incongruence on commitment and code contribution activity in open source software communities. *MIS Quarterly*, 43(3), 951–983.

- Nagle, F. (2018). Learning by contributing: Gaining competitive advantage through contribution to crowdsourced public goods. *Organization Science*, 29(4), 569–587.
- Nam, D., Macvean, A., Hellendoorn, V., Vasilescu, B., & Myers, B. (2024). Using an llm to help with code understanding. *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–13.
- Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654), 187–192.
- OpenAI. (2024). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub copilot. *arXiv preprint arXiv:2302.06590*.
- Replit. (2023). Replit and Google cloud partner to advance generative AI for software development [Accessed August 19, 2024]. <https://blog.replit.com/google-partnership>
- Rolandsson, B., Bergquist, M., & Ljungberg, J. (2011). Open source in the firm: Opening up professional practices of software development. *Research Policy*, 40(4), 576–587.
- Rubinstein, J., Meyer, D. E., & Evans, J. E. (2001). Executive control of cognitive processes in task switching. *Journal of Experimental Psychology: Human Perception and Performance*, 27, 763–797.
- Sarkar, A., Gordon, A. D., Negreanu, C., Poelitz, C., Ragavan, S. S., & Zorn, B. (2022). What is it like to program with artificial intelligence? *arXiv preprint arXiv:2208.06213*.
- Shah, S. K. (2006). Motivation, governance, and the viability of hybrid forms in open source software development. *Management Science*, 52(7), 1000–1014.
- Shihab, E., Kamei, Y., Adams, B., & Hassan, A. E. (2013). Is lines of code a good measure of effort in effort-aware models? *Information and Software Technology*, 55(11), 1981–1993.
- Song, F., Agarwal, A., & Wen, W. (2023). The impact of generative ai on collaborative open-source software development: Evidence from github copilot. *Available at SSRN 4856935*.
- Spadini, D., Aniche, M., & Bacchelli, A. (2018). Pydriller: Python framework for mining software repositories. *The 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- Svanberg, M., Li, W., Fleming, M., Goehring, B., & Thompson, N. (2024). Beyond ai exposure: Which tasks are cost-effective to automate with computer vision? *Available at SSRN 4700751*.
- Teng, D., Ye, C., & Martinez, V. (2024). Beyond orchestration: A resource altruism perspective towards ecosystem leadership. *Available at SSRN 4891683*.
- Tschang, F. T., & Almirall, E. (2021). Artificial intelligence as augmenting automation: Implications for employment. *Academy of Management Perspectives*, 35(4), 642–659.
- von Hippel, E., & von Krogh, G. (2003). Open source software and the “private-collective” innovation model: Issues for organization science. *Organization Science*, 14(2), 209–223.
- von Krogh, G., Haefliger, S., Spaeth, S., & Wallin, M. W. (2012). Carrots and rainbows: Motivation and social practice in open source software development. *MIS Quarterly*, 36(2), 649–676.

- von Krogh, G., & von Hippel, E. (2006). The promise of research on open source software. *Management Science*, 52(7), 975–983.
- Weber, S. (2005). The political economy of open source software and why it matters. In *Digital formations: It and new architectures in the global realm* (pp. 178–212). Princeton University Press.
- Xu, F., Medappa, P. K., Tunc, M. M., Vroegindewij, M., & Fransoo, J. C. (2025). Ai-assisted programming may decrease the productivity of experienced developers by increasing maintenance burden. *Available at SSRN 5521379*.
- Yeverechyahu, D., Mayya, R., & Oestreicher-Singer, G. (2024). The impact of large language models on open-source innovation: Evidence from github copilot. *arXiv preprint arXiv:2409.08379*.

Tables

Table 1: Variable Definitions and Summary Statistics

Variable	Definition	Mean	Std.Dev.
Dependent Variables			
Commits	Number of commits made by the developer in a given week	1.93	9.05
Repos Committed	Number of unique repositories that the developer contributed to in a given week	0.37	0.94
<i>Productivity Metrics (per commit)</i>			
Line Changes	Average number of lines changed (additions + deletions) per commit in a given week	539.28	2949.29
Meaningful Line Changes	Average number of meaningful lines changed per commit in a given week. Same as <i>Line Changes</i> , but excluding non-meaningful lines (e.g., whitespace-only, comments, or formatting)	76.95	267.34
Diff Delta	Average cognitive effort for commits in a week, where each commit’s score estimates effort, upweights substantial updates/deletions (especially to legacy code), downweights new / boilerplate lines, and penalizes short-term churn (lines quickly reworked or removed)	551.76	8952.40
Modified Files	Average number of files modified per commit in a given week	14.45	145.95
<i>Development Quality</i>			
Code Churn %	Percentage of code added in the week that is modified or removed within the subsequent 2 weeks	2.33	9.76
Copy-pasted %	Percentage of code that appears to be copy-pasted (e.g., duplicated blocks, low edit distance with external sources, or repeated snippets within the repo)	6.48	13.05
Project Novelty	Mean cosine distance between embeddings of repository descriptions for repos created in the week and embeddings of the developer’s existing repos (repos created within the 12 months prior to the observation window)	0.73	0.11
<i>Language Portfolio</i>			
Languages Added	Number of distinct programming languages first observed in a given week (relative to all previously seen languages)	0.01	0.18
<i>Work Cadence</i>			
Sessions	Average number of commit-activity clusters in a given week; a new session begins after ≥ 4 hours of inactivity	0.23	1.03
Session Duration	Average duration (in minutes) of sessions within the week	30.97	64.64
Session Intermission	Average gap (in hours) between consecutive sessions	12.10	178.36
Work Day Dist. Std.	Standard deviation of the distribution of work sessions within a week	0.59	0.77
Weekend Commit %	Share of commits that occur on Saturday or Sunday	0.18	0.33
Control Variables			
AvgRepoAge	Average age (in weeks) of repositories the developer contributed to in a given week	9.35	42.77
AccountAge	Number of weeks since the developer’s account creation	328.40	167.22
Observations	36450		
Number of Users	1350		

Note. The descriptive statistics are calculated for matched samples, where developers were successfully matched on pre-treatment characteristics. Line ratio variables, such as Code Churn %, are calculated in relation to all the lines added during the week. All variables are constructed at the user-week level.

Table 2: Developer Activities

	(1) Ln(Commits)	(2) Ln(Commits)	(3) Ln(Repos)
Pro $\times$ PostAnnouncement	0.10*** (0.04)	0.09** (0.03)	0.05*** (0.02)
Pro $\times$ PostLaunch	0.13*** (0.04)	0.12*** (0.04)	0.06*** (0.02)
L.Ln(AvgRepoAge)		0.07*** (0.01)	0.03*** (0.01)
Ln(AccountAge)		0.54 (0.44)	0.28 (0.21)
User FE	Yes	Yes	Yes
Week FE	Yes	Yes	Yes
N	36450	35100	35100
# Users	1350	1350	1350
Adj. R2	0.39	0.41	0.42

Note. Pro, PostAnnouncement, and PostLaunch are automatically dropped when including user and weekly fixed effects. L.Ln(AvgRepoAge) is the one-week lagged (logarithmized) mean age of repositories the user has worked on, where repository age is measured in weeks since creation. AccountAge refers to the number of weeks since creating the user account. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table 3: Commit-level Change

	(1) Ln(Line Changes)	(2) Ln(Meaningful Line Changes)	(3) Ln(Diff Delta)	(4) Ln(Modified Files)
Pro $\times$ PostAnnouncement	0.21 (0.16)	0.29* (0.15)	0.26 (0.18)	0.03 (0.09)
Pro $\times$ PostLaunch	0.28* (0.17)	0.49*** (0.15)	0.38** (0.18)	0.08 (0.09)
L.Ln(AvgRepoAge)	0.04** (0.02)	0.02 (0.02)	0.03 (0.03)	0.03* (0.01)
Ln(AccountAge)	0.07 (2.30)	-0.13 (2.15)	0.96 (2.19)	0.71 (1.07)
User FE	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes
N	4247	4247	4247	4247
# Users	635	635	635	635
Adj. R2	0.68	0.60	0.40	0.31

Note. The dependent variables are constructed at user-week level. Line Changes captures average lines modified per commit (additions plus deletions). File Mods measures average files modified per commit. All variables are logarithmized. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table 4: Work Hours and Session Analysis

	(1) Ln(Estimated Hours)	(2) Ln(Sessions)	(3) Session Duration, Ln(mins)
Pro $\times$ PostAnnouncement	0.14** (0.06)	0.12* (0.07)	0.22 (0.20)
Pro $\times$ PostLaunch	0.07 (0.06)	0.08 (0.06)	0.24 (0.17)
L.Ln(AvgRepoAge)	0.03*** (0.01)	0.02** (0.01)	0.07*** (0.02)
Ln(AccountAge)	0.70 (0.79)	0.73 (0.86)	2.45 (2.41)
User FE	Yes	Yes	Yes
Week FE	Yes	Yes	Yes
N	4247	4247	4247
# Users	635	635	635
Adj. R2	0.27	0.23	0.18

Note. Observations are restricted to user-weeks with at least one commit to track shifts in active work patterns. Sessions are commit-activity clusters, where a session begins after at least 240 minutes of inactivity. Raw session duration is computed as (time of last commit + 15 minutes) – (time of first commit – 30 minutes). We then applied adjustments on estimate session duration by commit frequency. Estimated Hours are estimated as the sum of all adjusted session hours whose start times fall within the week. All dependent variables are natural-log transformed. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table 5: Activity Distribution

	(1) Intermission, Ln(hrs)	(2) Work Day Dist. Std.	(3) Weekend Commit %
Pro $\times$ PostAnnouncement	0.22 (0.35)	0.10 (0.08)	0.00 (0.04)
Pro $\times$ PostLaunch	0.70*** (0.26)	0.12* (0.07)	0.02 (0.03)
L.Ln(AvgRepoAge)	0.01 (0.06)	0.03** (0.01)	-0.00 (0.00)
Ln(AccountAge)	-2.65 (3.43)	1.76 (1.46)	0.54 (0.33)
User FE	Yes	Yes	Yes
Week FE	Yes	Yes	Yes
N	2712	4247	4247
# Users	491	635	635
Adj. R2	0.37	0.31	0.16

Note. Intermission measures the average length of intermissions between sessions within a week; this regression is restricted to weeks with at least two sessions, since intermission is otherwise undefined. Work Day Dist. Std. is the standard deviation of the weekly session counts across workdays, capturing dispersion in day-by-day activity. Weekend Commit (%) is the share of a user's own commits made on Saturday or Sunday over total commits in that week. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table 6: Effects on Development Exploration and Quality Outcomes

	Technical Scope	Engineering Quality		Novelty
	(1) Poisson(New Langs)	(2) Code Churn %	(3) Copy-pasted %	(4) Project Novelty
Pro $\times$ PostAnnouncement	0.19 (0.23)	-0.00 (0.02)	0.02** (0.01)	-0.03 (0.03)
Pro $\times$ PostLaunch	0.52** (0.22)	0.01 (0.01)	0.02* (0.01)	-0.01 (0.02)
L.Ln(AvgRepoAge)	0.14*** 0.04	-0.00** (0.00)	0.00 (0.00)	0.00 (0.00)
Ln(AccountAge)	1.03 (1.58)	0.04 (0.10)	0.05 (0.14)	-0.28 (0.20)
User FE	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes
N	7150	2476	4247	756
# Users	635	229	392	275
Adj. R^2 / Log Likelihood	-1544.33	0.10	0.30	0.47

Note. Poisson(Langs Added) models the count of distinct programming languages first observed in the week using xtpoisson; xtpoisson does not report an adjusted R^2 , so we report the model log likelihood instead. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table 7: Heterogeneous Effects by Pre-treatment Commit Intensity

	(1) Ln(Commits)	(2) Ln(Repos)	(3) Ln(Work Hours)	(4) Ln(Sessions)	(5) Poisson(Langs Added)	(6) Project Novelty
Pro $\times$ PostAnnouncement	0.16*** (0.04)	0.07*** (0.02)	0.37*** (0.13)	0.41*** (0.15)	1.05** (0.46)	-0.05 (0.03)
Pro $\times$ PostAnnouncement $\times$ PastAvgCommits	-0.24** (0.11)	-0.09** (0.04)	-0.43* (0.23)	-0.53** (0.25)	-1.86** (0.87)	0.09** (0.04)
Pro $\times$ PostLaunch	0.30*** (0.04)	0.15*** (0.02)	0.39*** (0.13)	0.41*** (0.13)	2.26*** (0.42)	-0.06** (0.03)
Pro $\times$ PostLaunch $\times$ PastAvgCommits	-0.60*** (0.10)	-0.27*** (0.04)	-0.62*** (0.23)	-0.63*** (0.21)	-4.37*** (0.83)	0.08** (0.04)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	35100	35100	4247	4247	7150	1293
# Users	1350	1350	635	635	275	367
Adj. R2 / Log Likelihood	0.41	0.42	0.28	0.24	33801.7	0.42

Note. PastAvgCommits refers to the average monthly commits in the three months before CX's announcement. Poisson(Langs Added) models the count of distinct programming languages first observed in the week; xtpoisson does not report an adjusted R^2 , so we report the model log likelihood instead. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table 8: Summary of Robustness Tests

Test	Description	Issues Mitigated
Pre-trend test	Conducted a relative time model to assess temporal trends and parallel patterns (Figure A4-A5)	Parallel trend assumption
Try alternative matching specifications	Add variables that depicts users’ historical development pattern and users’ country to our CEM matching process (Table A5); Replicated results using samples obtained through 1-on-1 PSM (Table A6)	Self-selection bias
Limit repositories to Generative-AI-related projects	We classified repositories using available text information and replicated the results on overall development activities (Table A17)	Alternative explanation: concurrent surge in generative AI-related projects
Try alternative regression models	Use $\log(n+1)$ transformation as a substitute for Poisson count model (Table A18); Use Logit and linear probability model for binary career outcome analyses (Table A19-A20)	Model mis-specification
Try alternative time spans for variable construction	Measure users’ past activity level based on past one year’s commit volume (Table A4).	Measurement sensitive to time span selection

Table 9: Career Outcomes (Probit)

	In 6 Months			In 12 Months		
	(1) Job Hopping	(2) Internal Transition	(3) Promotion	(4) Job Hopping	(5) Internal Transition	(6) Promotion
Pro	-0.02 (0.15)	0.50** (0.20)	0.46** (0.21)	0.03 (0.14)	0.46*** (0.18)	0.31* (0.18)
Managerial Role	-0.44** (0.21)	-0.72** (0.31)	0.00 (.)	-0.23 (0.17)	-0.42* (0.24)	-0.65** (0.29)
Employment Type	Yes	Yes	Yes	Yes	Yes	Yes
Education	Yes	Yes	Yes	Yes	Yes	Yes
Industry	Yes	Yes	Yes	Yes	Yes	Yes
Company Size	Yes	Yes	Yes	Yes	Yes	Yes
N	445	379	293	460	408	398
Chi2	23.41	35.29	24.25	36.05	48.35	36.13
Pseudo R2	0.07	0.17	0.15	0.08	0.16	0.15

Note. This table reports the effect on users' career transitions within 6 and 12 months after treatment. The dependent variables capture different types of career movements: "Job Hopping" indicates switching to a different employer; "Internal Transition" represents any role change within the same company; "Promotion" indicates upward movement in job level or responsibilities. All models are probit regressions and include controls for employment type (full-time, part-time, contractor), education level, industry sector, and company size. Coefficients represent marginal effects. Robust standard errors are shown in parentheses. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Figures

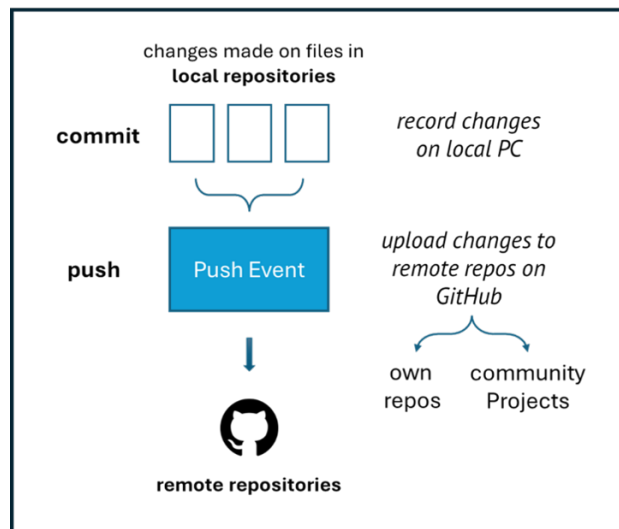


Figure 1: Flowchart of the process of using Git for coding activities

Note. A developer will first make changes to a project and then will commit these changes to the project. Next, once they are happy with the changes on their local computer they will then push this to the remote repository on GitHub.

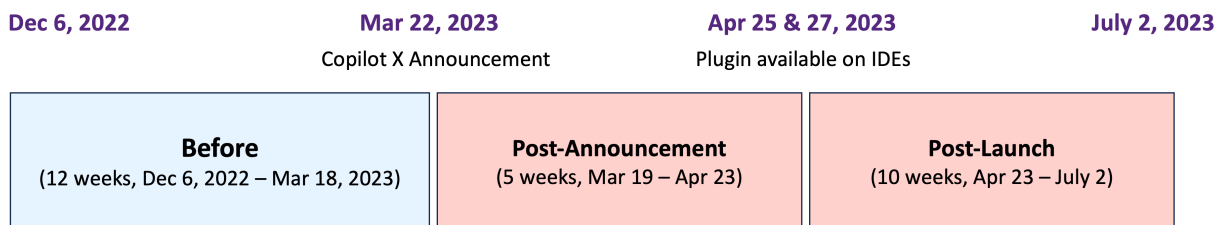


Figure 2: CX Launch Timeline

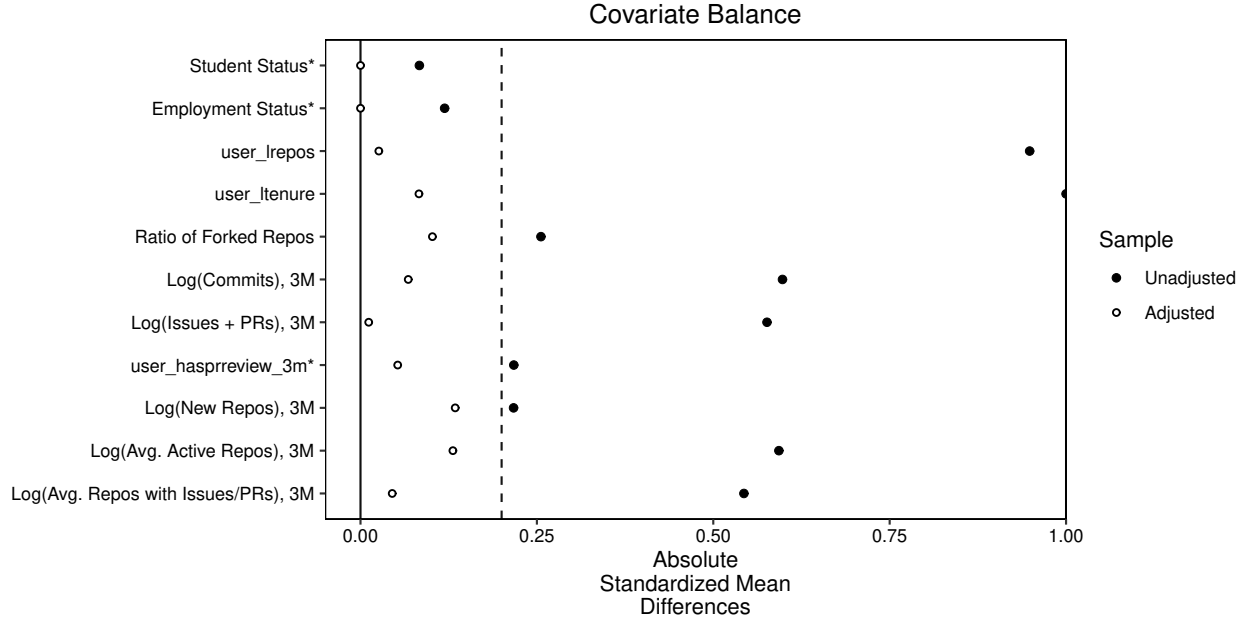


Figure 3: Covariate Balance Before/After CEM

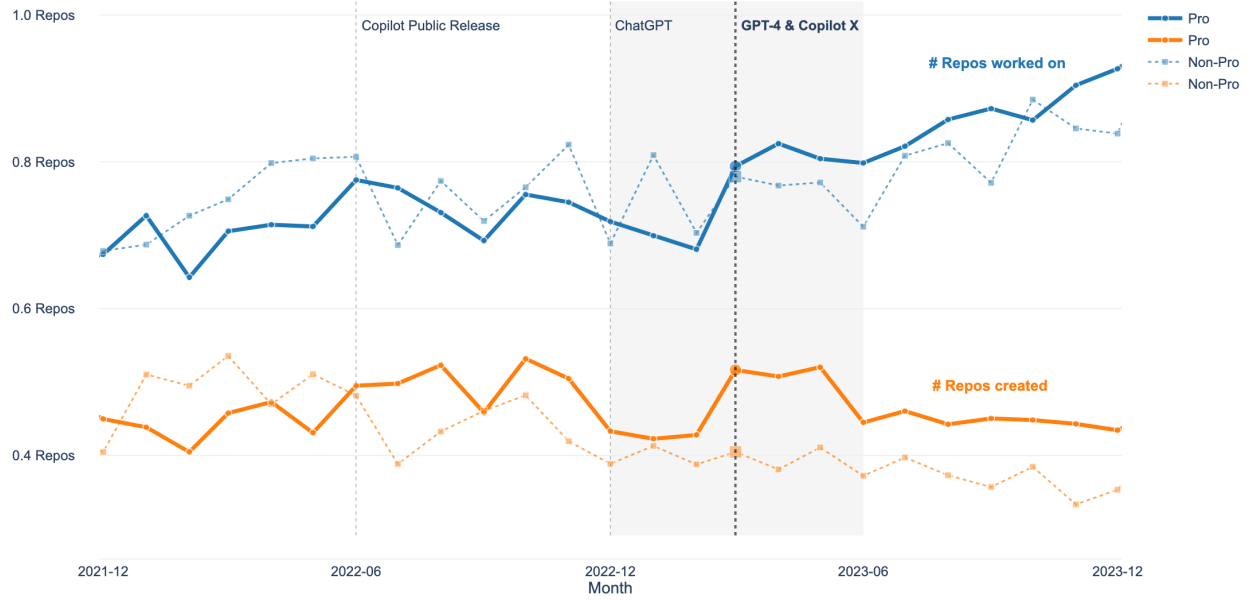


Figure 4: Monthly Repository Creation and Working Repositories

Note. The figure plots monthly averages of repositories created (orange lines) and worked on (blue lines) for Pro users (solid lines) and non-Pro users (dashed lines). Vertical lines mark major technological events. The shadowed interval indicates the observation window in our main analyses. Working on a repository is defined as making at least one commit to it in a given month.

Appendix 1 Programming Task Characteristics and the Evolution of AI Coding Assistants

Open source software (OSS) development encompasses both initial project creation and ongoing iterative development (see Figure A1). When initiating a new project, developers begin by generating an idea for a software solution and establishing a repository to house the codebase—steps that occur once per project. The subsequent development process, however, follows an iterative cycle of implementation, documentation, and collaboration that repeats throughout the project’s lifecycle. Within this iterative cycle, developers generate ideas for new features or improvements to the existing codebase, then implement these ideas through programming. The coding phase—historically the most cognitively demanding—requires developers to understand existing code architecture, recall syntax and language conventions, consult documentation, and write new code that integrates with the existing codebase. Once satisfied with their changes, developers commit these modifications locally before pushing them to the remote repository, making their work visible to the broader community. External contributors who lack direct write access may propose changes through pull requests, which project maintainers review and potentially merge. This cycle of ideation, implementation, commitment, and collaboration repeats continuously as the project evolves, with developers regularly returning to add features, fix bugs, or refactor existing code within the same repository.

This workflow imposed significant cognitive costs, particularly during the code ideation phase. Developers faced several characteristic challenges. (1) Context acquisition costs: Understanding unfamiliar codebases required substantial time reading documentation, tracing dependencies, and comprehending architectural patterns. (2) Syntax and convention recall: Each programming language demands mastery of specific syntax, idioms, and best practices that must be retrieved from memory or external references. (3) Context switching penalties: Moving between projects or returning to dormant work required re-establishing mental models of code structure and project state. (4) Documentation search costs: Finding relevant API documentation, library references, and code examples consumed substantial development time. Overall, these characteristics created a “few attempts, long duration” working pattern, where developers invested extended continuous sessions to overcome initial comprehension barriers and maintain productive flow states.

The First Wave: GitHub Copilot (2021)

Launched in 2021, GitHub Copilot introduced AI-powered code autocompletion based on OpenAI’s Codex model. This first-generation tool operated primarily at the line and function level, suggesting code completions as developers typed. While revolutionary, this assistance remained reactive and localized—developers still needed to know what to write and where, with Copilot accelerating the mechanical aspects of code production. The impact of the initial Copilot release on programming task characteristics was meaningful but bounded. Studies documented that developers completed tasks up to 55% faster when using Copilot, with particularly strong gains in reducing time spent on boilerplate and repetitive code patterns (Kalliamvakou, 2022; Peng et al., 2023). Developer surveys

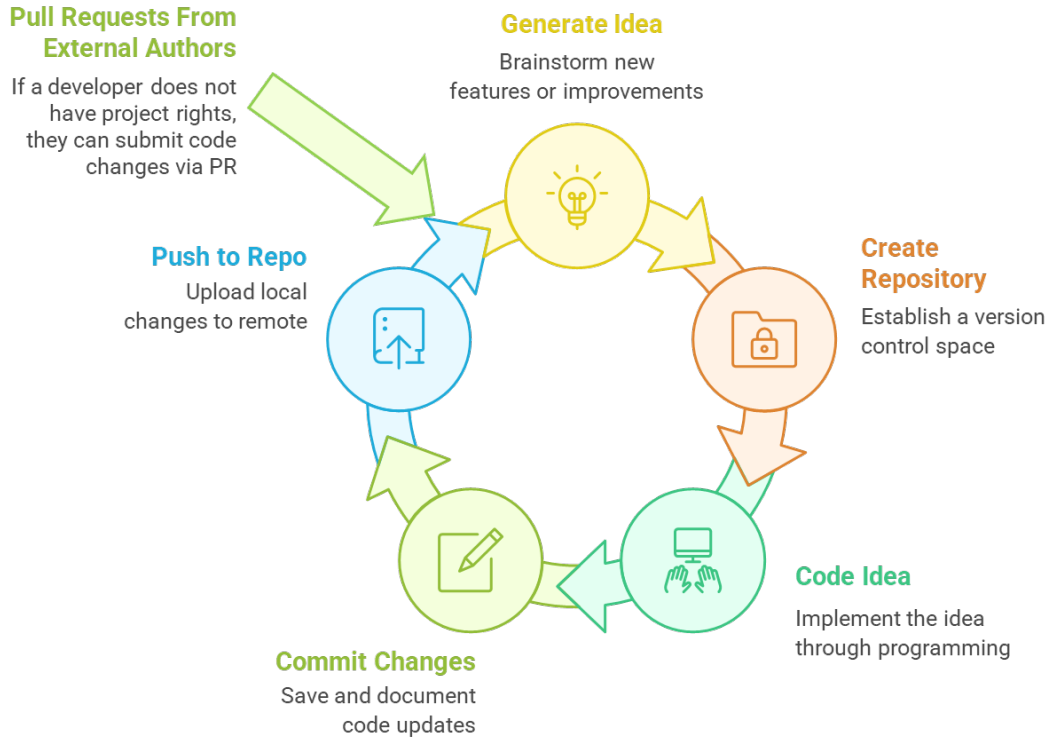


Figure A1: OSS Development Before AI Coding Assistants

revealed that 87% of users reported Copilot preserved mental effort during repetitive tasks, and 73% reported staying in a "flow state" more consistently when using the tool (Kalliamvakou, 2022). Copilot also accelerated syntax recall for familiar operations by providing in-context suggestions, reducing the need to consult external documentation—a notable benefit given that 63% of developers report spending more than 30 minutes daily searching for answers or solutions (Dembovsky, 2023). However, the fundamental requirement for developers to hold project context in working memory remained unchanged, as the original Copilot lacked conversational memory and multi-file awareness; each suggestion was generated independently based on immediate local context without reference to broader architectural understanding or prior interactions within the same session.

The Transformation: GitHub Copilot X (2023)

The March 2023 announcement and April 2023 release of GitHub Copilot X represented a qualitative shift in AI coding assistance. Powered by GPT opposed to Codex, one of CX's tools introduced conversational interaction through an embedded chat interface (see Figure A2, fundamentally altering how developers could approach programming tasks. Several key capabilities distinguish CX from its predecessor and transform the characteristics of programming tasks. First, natural language interaction enables developers to describe their intentions conversationally rather than through code, substantially lowering barriers for unfamiliar tasks or programming languages. Second, multi-file context awareness allows CX to incorporate the active file and open tabs into its

working context, enabling reasoning across a developer’s immediate workspace rather than being limited to single-file suggestions. Third, code explanation and comprehension features permit developers to request explanations of existing code, dramatically reducing the context acquisition costs traditionally incurred when navigating unfamiliar repositories. Finally, test generation and debugging support allows CX to generate unit tests (as shown in Figure A2) and propose fixes for identified bugs, streamlining quality assurance tasks that previously required substantial manual effort. Thus, CX has the capabilities to externalize context maintenance by summarizing code states, reducing the burden of holding complex structures in working memory, and lowers language barriers through natural language mediation, enabling developers to describe intent rather than recall syntax in unfamiliar languages.

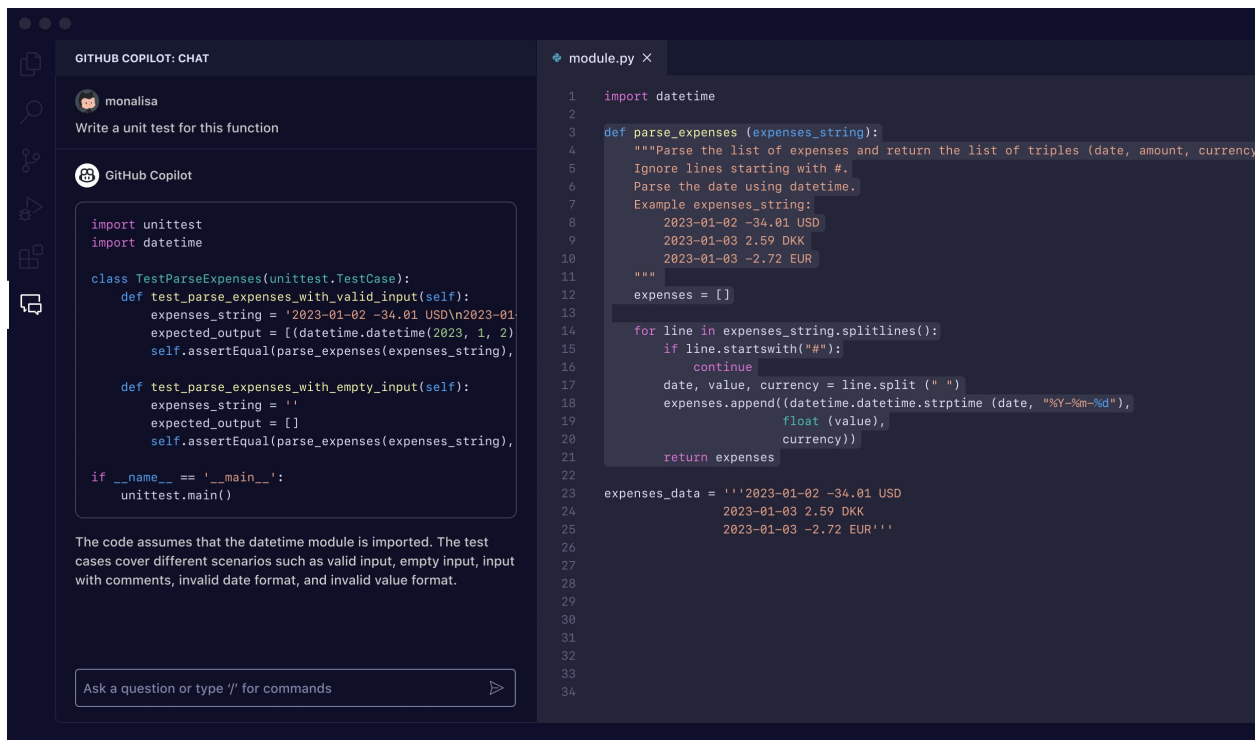


Figure A2: Screenshot of GitHub Copilot Extension in Visual Studio Code

Appendix 2 Public Interests on GitHub Copilot X

Figure A3 shows the weekly search interest for “GitHub Copilot” and “Copilot” worldwide from June 2022 to June 2023. The modest lift in interest after the initial public release of GitHub Copilot in 2022 contrasts with a pronounced spike around the March 2023 announcement of CX, reflecting a more hyped anticipation from the public on the release of the suite of ‘X’ tools from GitHub Copilot.

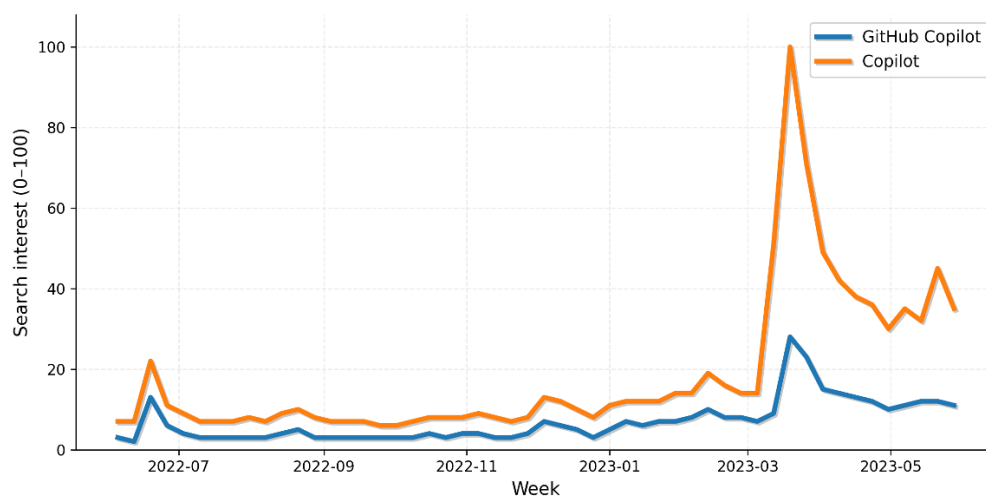


Figure A3: Screenshot of Google trends for the search term ‘github copilot’ and ‘copilot’

Appendix 3 Related Studies on GitHub Copilot

Table A1 provides an overview of key studies examining the effects of GitHub Copilot on OSS and compares the present studies positioning within this literature.

Table A1: Related Works on GitHub Copilot

		This Study	Peng et al. (2023)	Cui et al. (2024)	Hoffmann et al. (2024)	Yeverehyahu et al. (2024)	Song et al. (2023)	Xu et al. (2025)	Baird et al. (2024)
Version Release*		CX	C	C	C	C	C	C	C
Level of Analysis	Developer	✓		✓	✓		+	✓	
	Project	+	✓			✓	✓	✓	
	Company								✓
Developer Activities & Characteristics	Events	✓		✓	✓	✓	✓	✓	
	Technical Indicators	✓							
	Developer Heterogeneity	✓	✓		✓		✓		
Labor Market Outcome	Developer	✓							
	Company								✓
Method	Empirical Model	✓		✓	✓	✓	✓	✓	✓
	Controlled / Qualitative		✓						

Note. *C = Copilot, CX = Copilot X. Events are any metrics classed within the GitHub API events classification. Technical indicators refer to data examining coding activities at a deeper level, such as details in GitHub diff files. “+” denotes a supplementary analysis used to inform the main project findings.

Appendix 4 Tests on Parallel Trend Assumption

To assess the plausibility of the parallel-trends assumption in our difference-in-differences design, Figure A4-A5 present the estimated results from event-time regressions of development activities on week indicators that lead and lag the treatment week, estimated with the same user and calendar-week fixed effects as in the baseline specification. The coefficients on the pre-treatment dummies (`pre11`–`pre2`) are small in magnitude and rarely statistically significant, providing little evidence of systematic differential pre-trends between treated and control users. Although the coefficients for weeks `pre6` and `pre7` are significantly negative in one specification, a joint test of all pretreatment coefficients fails to reject the null of no systematic difference between treatment and control groups prior to CX exposure.

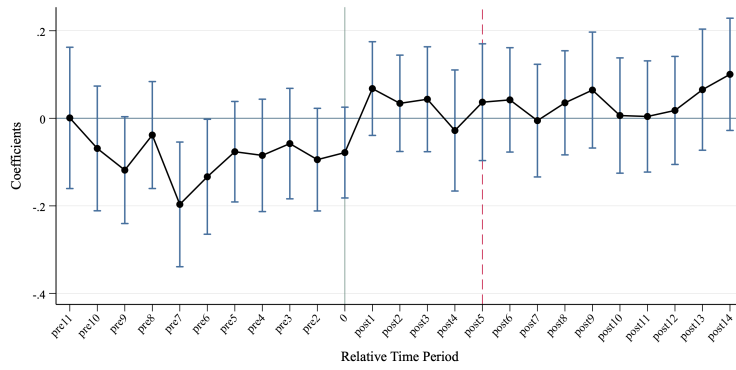


Figure A4: Pretrend test on Ln(Commits)

Note. Event-time coefficients are estimated from a regression that omits `pre1` as the reference period. A joint F-test of all pretreatment coefficients yields an F-statistic of 1.19 with a p-value of 0.29. Point estimates are plotted with 95% confidence intervals.

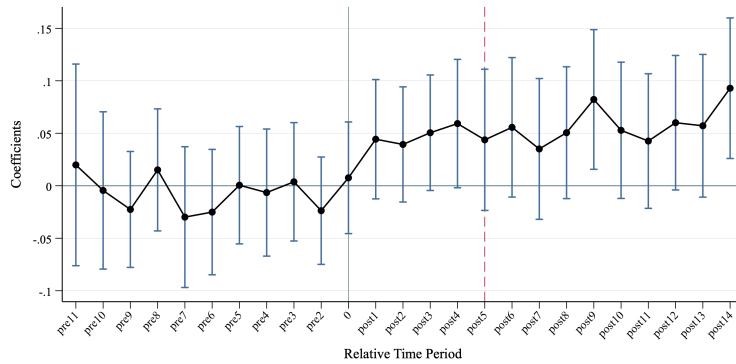


Figure A5: Pretrend test on Ln(Repos)

Note. Event-time coefficients are estimated from a regression that omits `pre1` as the reference period. A joint F-test of all pretreatment coefficients yields an F-statistic of 0.51 with a p-value of 0.88. Point estimates are plotted with 95% confidence intervals.

Appendix 5 Additional Heterogeneous Analysis

In this section, we present additional heterogeneous analysis conducted on variables from the main tables within the initial main analysis. Table A2 illustrates the heterogeneous effects of users on code contributions, while Table A3 presents the results of user heterogeneity in relation to work patterns. Furthermore, we append the heterogeneous results when using the total number of historical commits each user made in the past year as a substitute for user commit intensity, serving as a robustness check (Table A4).

Table A2: Heterogeneous Effects on Code Contributions by Pre-treatment Commit Intensity

	(1) Ln(Line Changes)	(2) Ln(Meaningful Line Changes)	(3) Ln(Diff Delta)	(4) Ln(Modified Files)
Pro × PostAnnouncement	0.44 (0.34)	0.59* (0.33)	0.27 (0.35)	0.21 (0.17)
Pro × PostAnnouncement × PastAvgCommits	-0.44 (0.56)	-0.55 (0.51)	-0.03 (0.54)	-0.34 (0.27)
Pro × PostLaunch	0.46 (0.35)	0.99*** (0.28)	0.40 (0.31)	0.15 (0.17)
Pro × PostLaunch × PastAvgCommits	-0.34 (0.72)	-0.97* (0.52)	-0.03 (0.56)	-0.14 (0.33)
Controls	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes
N	4247	4247	4247	4247
# Users	635	635	635	635
Adj. R2	0.68	0.60	0.40	0.31

Note. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table A3: Heterogeneous Effects on Work Patterns by Pre-treatment Commit Intensity

	(1) Session Duration, Ln(mins)	(2) Intermission, Ln(hrs)	(3) Work Day Dist. Std.	(4) Weekend Commit %	(5) Copy- pasted %	(6) Code Churn %
Pro × PostAnnouncement	0.98*** (0.36)	1.11 (0.76)	0.24** (0.12)	-0.02 (0.07)	0.02 (0.02)	0.04 (0.04)
Pro × PostAnnouncement × PastAvgCommits	-1.44** (0.61)	-1.52 (1.13)	-0.25 (0.24)	0.06 (0.11)	0.01 (0.04)	-0.07 (0.05)
Pro × PostLaunch	0.95*** (0.32)	1.93*** (0.67)	0.47*** (0.10)	0.10* (0.06)	0.02 (0.02)	0.01 (0.02)
Pro × PostLaunch × PastAvgCommits	-1.37** (0.61)	-2.21** (1.04)	-0.69*** (0.19)	-0.17* (0.09)	0.00 (0.03)	0.03 (0.03)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	4247	2712	4247	4247	4251	2623
# Users	635	473	635	635	635	399
Adj. R2	0.18	0.23	0.31	0.16	0.30	0.09

Note. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table A4: Heterogeneous Effects by Pre-treatment Commit Intensity (Past 1 Year)

	(1) Ln(Commits)	(2) Ln(Repos)	(3) Ln(Work Hours)	(4) Ln(Sessions)	(5) Langs Added	(6) Project Novelty
Pro × PostAnnouncement	0.08** (0.04)	0.04** (0.02)	0.19* (0.11)	0.12 (0.12)	0.66* (0.38)	-0.03 (0.03)
Pro × PostAnnouncement × PastCommitsly	0.01 (0.12)	0.01 (0.05)	-0.09 (0.20)	0.02 (0.22)	-1.22 (0.79)	0.06 (0.05)
Pro × PostLaunch	0.19*** (0.05)	0.10*** (0.02)	0.21** (0.09)	0.20** (0.10)	1.12*** (0.35)	-0.04* (0.02)
Pro × PostLaunch × PastCommitsly	-0.23** (0.10)	-0.10** (0.04)	-0.29* (0.18)	-0.25 (0.18)	-1.59** (0.71)	0.05 (0.04)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	35100	35100	4247	4247	7150	1293
# Users	1350	1350	635	635	275	367
Adj. R2 / Log Likelihood	0.41	0.42	0.27	0.23	-1541.60	0.42

Note. Standard errors are clustered at the user level. Again, we reports poisson estimation results for variable “Langs Added”. Log likelihood is reported for this model as a substitute of adjusted R square. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Appendix 6 Robustness Check Using Different Matching Specifications

To assess the robustness of our main findings to alternative matching specifications, we re-estimate our core models using two different approaches. Table A5 improves upon existing matching criteria by considering demographic factors (developer’s resident country), and historical working patterns (code maintenance and work cadence), to ensure a balanced distribution of co-variates between the treatment and control groups. Table A6 employs a one-to-one matching approach without replacement propensity score matching (PSM). This alternative specification helps ensure that our results are not influenced by the specific matching method used in our main analysis.

Table A5: Regression analysis using stricter matched samples

	(1) Ln(Commits)	(2) Ln(Repos)	(3) Ln(Work Hours)	(4) Session In- termission, Ln(hours)	(5) Poisson(Langs Added)	(6) Project Novelty
Pro × PostAnnouncement	0.09** (0.04)	0.03* (0.02)	0.10 (0.09)	0.61 (0.45)	0.02 (0.94)	-0.03 (0.03)
Pro × PostLaunch	0.10** (0.05)	0.04** (0.02)	0.17** (0.07)	0.95** (0.40)	0.81*** (0.00)	-0.03 (0.02)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	22568	22568	2400	1524	4628	667
# Users	868	868	380	282	178	199
Adj. R2 / Log Likelihood	0.41	0.41	0.27	0.22	-1011.1	0.43

Note. In this specification, we also took into account users’ observed differences in their resident countries and historical development patterns, such as the number of active weeks, the emphasis on maintenance and new feature development, and the work cadence (including the number of weekly development sessions and the average duration of each session). Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table A6: Regression Analysis Using PSM Matched Samples (1-to-1 without Replacement)

	(1) Ln(Commits)	(2) Ln(Repos)	(3) Ln(Work Hours)	(4) Session In- termission, Ln(hours)	(5) Poisson(Langs Added)	(6) Project Novelty
Pro $\times$ PostAnnouncement	0.10*** (0.04)	0.05*** (0.02)	0.01 (0.06)	0.20 (0.24)	-0.13 (0.27)	-0.00 (0.01)
Pro $\times$ PostLaunch	0.09** (0.04)	0.04** (0.02)	0.02 (0.06)	0.34 (0.21)	0.49** (0.24)	-0.01 (0.01)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	27508	27508	4921	3235	5278	1413
# Users	1058	1058	649	498	203	362
Adj. R2 / Log Likelihood	0.51	0.53	0.32	0.21	-1110.31	0.53

Note. The specification of PSM is one-on-one matching without replacement on the same batch of co-variates. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Appendix 7 AI Detection of Users’ Code Additions During the Study Period

Data Acquisition and Preparation

In order to systematically verify our Pro status as an appropriate proxy for those who have access to CX, we decided to build and run a bespoke AI-Human code classification model and then classify the code additions in the samples’ GitHub repository diff files during the observational period. While commercial tools are available for such tasks, they do not allow for much flexibility both in terms of time and costs. Furthermore, internal testing suggested most have a proclivity to raise false positives in favor of AI despite feeding known human-written code examples. The construction of the dataset for discerning human-written code from AI-generated code involved a multi-stage pipeline designed to ensure code quality, eliminate redundancy, and strictly enforce class balance across programming languages and problem definitions. The source of human-generated code was the IBM CodeNet dataset (<https://github.com/IBM/Project.CodeNet>), a large-scale collection of code submissions to an online judging systems collected in 2021. AI-generated code was produced using various open-source LLMs prompted to solve the same set of problems found in the IBM CodeNet dataset. To prevent the classifier from overfitting to the stylistic quirks of a single generator, AI code was synthesized using a diverse ensemble of open-source LLMs (i.e., CodeGemma, CodeLlama, DeepSeek, Codestral). This ensemble included several foundational code-centric and reasoning models with varying parameter sizes (from 7-billion to 30-billion), ensuring a broad representation of machine-generated logic and syntax. Due to the large number of generated human data and access to a server-grade GPU for one-month only (RTX A5000), we were able to produce enough AI-generated samples leading to a class imbalance, with Human samples outnumbering AI samples by a ratio of approximately 13.4 : 1 (7.46×10^6 vs. 5.57×10^5). To mitigate bias toward the majority class, we implemented a hierarchical, stratified sampling algorithm to construct a perfectly balanced dataset. The sampling logic operated on the principle of Exact Count Matching. For every unique combination of Problem P and Programming Language L present in the AI dataset, we calculated the exact number of AI-generated files, denoted as N_{AI} . Our goal was to select exactly N_{AI} human files to match this cluster. The selection of human files followed a strict priority hierarchy to maximize semantic relevance:

1. **Primary Match (Problem + Language):** We first attempted to draw N_{AI} files from the human dataset that matched both the specific problem ID and the programming language.
2. **Fallback 1 (Same Problem, Any Language):** If exact matches were insufficient, the pool was widened to include human solutions for the same problem P written in *any* language. This preserved the semantic logic of the problem.
3. **Fallback 2 (Global, Same Language):** If the problem-specific pool was still empty, we selected files from the global dataset that matched the target language L , preserving syntactic

structure.

4. **Fallback 3 (Global, Any Language):** In the rare event no matches were found, we sampled from the global pool of all available human code.

Following the balancing phase, the final dataset consisted of 1,113,780 total samples, with a perfect 50/50 split between Human and AI classes. The data was partitioned into Training, Validation, and Test sets using a stratified split to preserve the class ratio across all subsets (Training Set (70%): 779,646 samples; Validation Set (15%): 167,067 samples; Test Set (15%): 167,067 samples). Table A7 illustrates the language distribution in the final processed dataset.

Table A7: Final Balanced Dataset Distribution by Language

Language	Samples (AI)	Samples (Human)
Python (.py)	46,434	46,434
C (.c)	46,423	46,423
Ruby (.rb)	46,423	46,423
C++ (.cpp)	46,416	46,416
Java (.java)	46,410	46,410
JavaScript (.js)	46,410	46,410
PHP (.php)	46,408	46,408
C# (.cs)	46,405	46,405
Rust (.rs)	46,393	46,393
OCaml (.ml)	46,391	46,391
Go (.go)	46,390	46,390
Scala (.scala)	46,387	46,387

Model Training and Selection

Following data preparation, we implemented a two-stage training pipeline designed to empirically identify the optimal transformer architecture for code classification. This process consisted of a preliminary ranking phase to filter candidate models, followed by a fine-tuning phase for the selected architecture. We evaluated a diverse set of pre-trained transformer models, ranging from general-purpose language models to domain-specific models pre-trained on large corpora of source code. The candidate pool included code-centric models: `codebert-base`, `graphcodebert-base`, `unixcoder-base`, and `CodeBERTa-small-v1`, and general purpose models: `roberta-base` and `distilroberta-base`.

To efficiently select the best performing model without expending computational resources on full training runs for every candidate, we implemented a "Ranking" epoch. We constructed a stratification subset consisting of 8,000 training samples and 2,000 evaluation samples. Each model was fine-tuned for exactly one epoch. The selection criterion was the Macro-F1 score on the validation subset. Following this evaluation, `CodeBERTa-small-v1` emerged as the superior

architecture, demonstrating the highest efficiency and predictive capability. The remaining models were subsequently dropped from further development. The winning model, **CodeBERTa-small-v1**, was subjected to a full training regimen on the complete dataset. The training configuration was engineered to maximize stability and generalization. We utilized the AdamW optimizer with a base learning rate $\eta_{max} = 2 \times 10^{-5}$ and weight decay $\lambda = 0.01$. To prevent catastrophic forgetting and ensure convergence, we implemented a learning rate schedule with a linear warmup followed by cosine decay. The model was trained for 8 epochs with a batch size of 24 and gradient accumulation steps of 2, resulting in an effective batch size of 48. The final deployed model, based on the **CodeBERTa** architecture, utilizes a 6-layer transformer encoder designed for lower latency inference while maintaining high representational capacity for syntactic structures. The architectural specifications are detailed in Table A8.

Table A8: Final Model Architecture

Hyperparameter	Value
Hidden Size (d_{model})	768
Num Hidden Layers	6
Num Attention Heads	12
Intermediate Size	3072
Hidden Activation	GELU
Vocabulary Size	52,000
Max Position Embeddings	514

Upon completion of training, the model was evaluated on the held-out Test set. The model achieved robust performance across all key classification metrics, indicating a strong ability to generalize to unseen code samples. The final performance metrics are presented in Table A9.

Table A9: Final Test Set Performance Metrics

Metric	Score
Accuracy	96.79%
F1-Score	96.87%
ROC-AUC	99.76%
Precision	94.58%
Recall	99.27%

Inference Pipeline and Deployment

Following the training and selection of the **CodeBERTa-small-v1** model, we architected a batch inference pipeline designed to process the large-scale repository of user activity. The objective of this phase was to ingest raw git diffs, extract relevant code modifications, and classify the authorship (Human vs. AI) for activity occurring within a specific temporal window. The inference pipeline was configured to process raw code data from the users’ diff files downloaded during the original data collection via PyDriller. To ensure the analysis remained relevant to the period of interest

(the pre/post-CX release) we enforced a strict temporal filter. Let C_u be the set of all commits associated with user u . We defined the active inference set C_{active} as:

$$C_{active} = \{c \in C_u \mid t_{start} \leq \text{timestamp}(c) \leq t_{end}\} \quad (2)$$

where $t_{start} = 2022-12-06$ and $t_{end} = 2023-07-02$.

A single git commit often comprises modifications to multiple files. To capture the full context of a user’s contribution, the pipeline decomposes each commit c into its constituent code changes. We implemented a custom parser, to isolate the generative signal from the version control metadata. For a given commit c , we extract a set of code segments $S_c = \{s_1, s_2, \dots, s_n\}$, where each segment corresponds to the additions made to a specific file. A segment s_i is retained if and only if it contains added lines of code (marked by “+”) and is not a binary or configuration file. Each extracted code segment $s \in S_c$ was tokenized using the pre-trained CodeBERTa tokenizer with a maximum sequence length of $L_{max} = 512$ tokens. To ensure deterministic handling of the extracted features, parallelism in the tokenizer was explicitly disabled. Next, the model computes a probability score $P(AI \mid s)$ for each segment individually. This granular approach allows the system to detect AI usage even in mixed commits where only specific files were generated by AI. The output of this phase is a structured record for every code modification, linking the probability score $P(AI)$ directly to its parent commit hash and filename. These raw probabilities serve as the basis for the commit-level aggregation performed in the subsequent statistical analysis. Lastly, the final classification label $\hat{y}$ is assigned based on a decision boundary of 0.5 (formally stated as Equation 3):

$$\hat{y} = \begin{cases} \text{AI} & \text{if } P(AI \mid x) \geq 0.5 \\ \text{Human} & \text{otherwise} \end{cases} \quad (3)$$

Empirical Verification of AI-Predicted Code

Utilizing the output from the AI prediction model, we conducted a statistical analysis of the classified commit history. The raw file-level predictions generated in the prior Phase were cleaned before being analyzed using clustered regression models. The raw dataset contained all file modifications identified in the diffs. To ensure the analysis focused on relevant coding activity, we applied the following exclusion criteria: (1) File type restriction: we retained only files corresponding to recognized programming languages. The allowed extensions included: `.py`, `.js/.ts`, `.java`, `.cpp`, `.c`, `.cs`, `.go`, `.rb`, `.php`, `.rs`, `.swift`, `.scala`, and others associated with core software development; (2) Documentation, configuration files, and assets were excluded; (3) Bot exclusion: to prevent automated processes from skewing the authorship signals, we filtered out commits where the author or committer name contained strings such as “bot”, “github actions”, or “github-actions”. Furthermore, since a single commit may contain multiple files with varying authorship probabilities, we aggregated predictions to the commit level. Let F_c be the set of code files modified in commit c . For each file $f \in F_c$, the model assigns a binary AI indicator $I_f \in \{0, 1\}$ based on the 0.5 probability

threshold. The commit-level AI share, S_c , can be formally written as Equation 4:

$$S_c = \frac{1}{|F_c|} \sum_{f \in F_c} I_f \quad (4)$$

We then defined a binary outcome variable Y_{it} for commit i at time t as:

$$Y_{it} = \begin{cases} 1 & \text{if } S_c \geq 0.5 \quad (\text{“AI-Generated”}) \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

We structured the analysis around the public release of CX to detect shifts in behavior. The timeline was divided into two primary observation windows. First was the pre-launch period ($T = 0$) where activity occurring from December 6, 2022, to April 22, 2023 was included. This encompasses the baseline usage prior to the public availability of the advanced CX features. The next period was the post-launch period ($T = 1$), which included activity occurring from April 23, 2023 (CX Public Release) to July 2, 2023. To quantify the relationship between user status (Pro vs. Non-Pro) and AI adoption, we employed Linear Probability Models (LPM) estimated via Ordinary Least Squares (OLS). Standard errors were clustered at the user level to account for serial correlation in commit behavior within individual users. We estimated the difference in AI adoption between Pro and Non-Pro users separately for the pre and post periods:

$$Y_{it} = \beta_0 + \beta_1 \text{Pro}_i + \epsilon_{it} \quad (6)$$

where Pro_i is a binary indicator equal to 1 if user i is a GitHub Pro member, and 0 otherwise.

To test the hypothesis that Pro users specifically increased their usage after the launch, we restricted the sample to Pro users ($\text{Pro}_i = 1$) and estimated:

$$Y_{it} = \alpha_0 + \alpha_1 \text{Post}_t + \nu_{it} \quad (7)$$

where Post_t is an indicator equal to 1 for activity in the Post-Launch period.

The analysis was performed on a final dataset of 40,909 verified commits across 1,333 distinct users. In the Pre-Launch period, Pro users showed a higher propensity for AI commits ($\beta_1 = 0.0955$), but this difference was not statistically significant ($p = 0.144$), suggesting a comparable baseline between groups. However, in the Post-Launch period, the gap widened ($\beta_1 = 0.1030$) and became statistically significant ($p = 0.035$). Furthermore, the longitudinal analysis confirms that Pro users significantly increased their AI adoption rate by approximately 3.75 percentage points ($\alpha_1 = 0.0375, p = 0.011$) following the release of Copilot X tools.

Table A10: Results of Comparing AI vs. Human Code Generation Likelihood
Pre/Post CX Public Launch

Specification	Coefficient	Estimate	Std. Err.	P-value
<i>Pre-Launch Baseline</i>	β_1 (Pro vs Non-Pro)	0.0955	0.065	0.144
<i>Post-Launch Analysis</i>	β_1 (Pro vs Non-Pro)	0.1030	0.049	0.035
<i>Within-Pro Growth</i>	α_1 (Pre $\rightarrow$ Post)	0.0375	0.015	0.011

Note. Standard errors clustered by username. Bold indicates statistical significance at $p < 0.05$.

Appendix 8 Motivations of Developers

To understand developers’ motivations for adopting Copilot, we conducted three complementary analyses. First, we examined all discussions within the GitHub Copilot discussion board, applying natural language processing techniques including sentiment analysis and topic modeling. This analysis allowed for us to reveal developers’ motivations and attitudes toward Copilot usage. Second, we constructed two moderating variables to examine heterogeneous treatment effects: student status and propensity for open source collaboration. We identified students through platform indicators such as .edu email addresses or self-identification in GitHub biographies. We measured open source collaboration propensity by analyzing each user’s complete repository portfolio and classifying projects as solo endeavors versus collaborative efforts involving multiple external contributors. Each of these analyses is described in detail in the following subsections.

GitHub Discussion Board Analysis

The initial corpus consisted of raw comments extracted from GitHub discussions regarding AI coding assistants. We scraped Copilot-related comments by collecting discussions under the “Copilot” topic panel on GitHub Community’s discussion board. We then appended the searched results using the keyword “copilot”. To ensure relevance, we only included discussions that were initiated after the announcement of GitHub Copilot. Following data collection, we applied multiple data preprocessing steps to ensure data quality. First, to mitigate noise generated by low-information content (e.g., “Thanks”, “+1”, or single-word responses), we filtered the corpus to exclude any comments with a character length of $L \leq 10$. Next, comment data was merged with user demographic profiles to incorporate the binary Pro status. Observations associated with users possessing undefined subscription statuses were removed, resulting in a final analytical corpus of $N = 6,998$ comments. Prior to topic modeling and sentiment analysis, raw textual data underwent normalization tailored to the specific requirements of each downstream task. For Latent Dirichlet Allocation (LDA), text was converted to lowercase, and noise artifacts—including URLs, user mentions (e.g., @username), and non-alphabetic characters—were removed. We constructed a custom stop-word list by combining standard English stopwords from the NLTK library with domain-specific tokens such as ‘github’, ‘copilot’, and ‘url’ to prevent these high-frequency terms from dominating topic definition.

To control for developer expertise, we calculated a continuous “Experience Index” (E_i) for each user based on historical activity data external to the comment dataset. Commit activity was aggregated over specific 3-month observation windows (e.g., Dec 2022 – Mar 2023). Because the distribution of commit counts was highly skewed, we applied a logarithmic transformation to the average 3-month commit count (the same approach used for the heterogeneous specification) (c) for each user, such that $c' = \ln(1 + c)$. This transformed value was subsequently normalized to a range of $[0, 1]$ using Min-Max scaling, as defined in Equation 8:

$$E_i = \frac{c'_i - \min(C')}{\max(C') - \min(C')} \quad (8)$$

where C' represents the vector of log-transformed commit counts for the entire user population.

We then employed LDA to categorize comments into latent thematic topics. The text corpus was vectorized using a CountVectorizer configured to capture both unigrams and bigrams ($n \in \{1, 2\}$). To refine the feature space, we filtered terms with a minimum document frequency of 3 and a maximum document frequency of 0.8, capping the vocabulary at the top 1,000 features. To determine the optimal number of topics (k), we trained multiple LDA models ranging from $k = 2$ to $k = 10$. The optimal solution was identified as $k = 9$, which maximized the C_v coherence score (Coherence: 0.5200). Comments were assigned to a dominant topic only if the model’s probability estimate exceeded a strict confidence threshold of $\rho \geq 0.3$; comments failing to meet this threshold were flagged as unassigned.

Next, we applied the VADER lexicon to measure sentiment in discussion posts. VADER has proven highly effective for analyzing informal online communication (Costello & Lee, 2022), making it well-suited for GitHub’s discussion environment. We structured the data to enable analysis of variance, with the VADER sentiment score serving as our dependent variable. Our primary independent variable was developer experience level, operationalized as a binary indicator distinguishing between casual users (those with limited GitHub activity) and active users (those with substantial development experience). To isolate the effect of experience from other discussion characteristics, we included several control variables: comment structure (main post versus reply, coded as a binary indicator where 1 = Main Post, 0 = Reply), comment length (total character count), word count, and temporal factors (year and month of posting). User subscription status (1 = Pro, 0 = Non-Pro) and the nine LDA topic categories were also incorporated to account for systematic differences in content and user characteristics.

We conducted a univariate analysis of variance (ANOVA) to examine the relationship between developer experience and sentiment. The model specification was:

$$Y_i = \beta_0 + \beta_1 X_i + \sum_j \gamma_j C_{ij} + \epsilon_i \quad (9)$$

where Y_i is the VADER sentiment score for comment i , X_i indicates developer experience level (Casual vs. Active), C_{ij} represents control variables including comment type, length metrics, temporal factors, and ϵ_i is the error term. The analysis revealed a statistically significant main effect of experience ($F(1, 710) = 3.92, p = .048$). As illustrated in Figure A6, casual developers exhibited significantly higher sentiment (estimated marginal mean = 0.20) compared to active developers (estimated marginal mean = 0.13) when discussing Copilot.

While the baseline analysis suggests experts are generally more critical, we performed a split-sample linear regression to determine if this effect is universal or context-specific. We regressed sentiment on experience within each latent topic individually, controlling for the same covariates.

The results (Table A12) indicate that the gap in experience is not uniform. We find a signif-

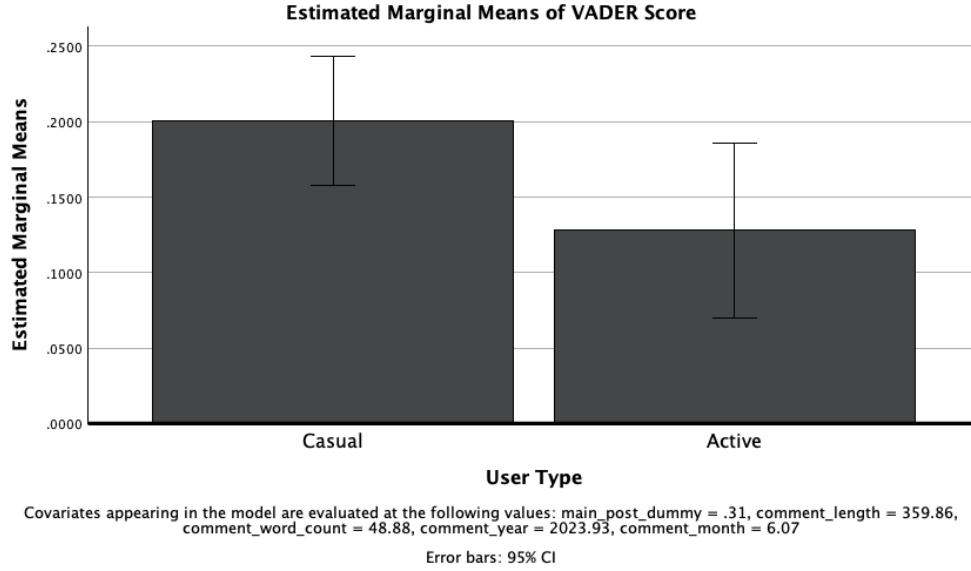


Figure A6: Estimated Marginal Means of Sentiment by Experience Level (Error Bars: 95% CI)

icant divergence in *Workflow Configuration* (Topic 1, $B = 0.255$, $p = 0.004$) and marginally in *Authentication & Access* (Topic 7, $B = 0.207$, $p = 0.076$). This pattern suggests that as users gain expertise, their expectations and pain points shift. While novices view the tool’s suggestions and settings as helpful additions, experts—who likely possess optimized workflows and complex development environments—are significantly more sensitive to friction in configuration and access reliability.

These topic-specific sentiment differences may reveal an important dimension of where AI coding assistants add value. The concentration of negative expert sentiment in workflow integration and authentication could indicate friction points that emerge when incorporating AI tools into sophisticated professional development contexts with established toolchains and security requirements. Conversely, casual users’ positive sentiment in these same areas is consistent with the tool functioning well in simpler, self-contained environments characteristic of learning-oriented and exploratory projects. This pattern aligns with our main behavioral findings whereby casual developers increase their coding activity and language exploration with CX, while experienced developers show minimal behavioral changes. Taken together, these patterns suggest that current ACAs may deliver greater benefits for self-directed learning and experimentation than for managing the technical complexity and workflow integration demands of large-scale OSS development.

Student and OSS Proclivity as Moderators

In addition to our primary moderation analysis based on user activity levels (Casual vs. Active), we conducted two supplementary heterogeneity analyses to ensure the robustness of our findings and rule out alternative explanations regarding user demographics and workflow styles. GitHub Pro status—our proxy for treatment—is provided free of charge to students in the GitHub Educational

Table A11: Baseline Analysis of Variance (Dependent Variable: Vader Sentiment)

Source	df	Mean Square	F
Corrected Model	6	.730	3.32
Intercept	1	.825	3.755
Main Post Dummy	1	.045	.207
Comment Length	1	2.458	11.19**
Comment Word Count	1	1.411	6.42*
Comment Year	1	.824	3.75
Comment Month	1	.001	.002
Experience (Low/High)	1	.862	3.92*
Error	710	.220	

Note. $R^2 = 0.027$ (Adjusted $R^2 = 0.019$). * $p < .05$, ** $p < .01$.

Table A12: Impact of Experience on Sentiment by Topic (Split-Sample Regression)

Topic	Label	Key Terms	Exp. B	S.E.	t	Sig.
-1	Outliers	n/a	-.318	.321	-.99	.367
0	Proxy & Server	error, proxy, node	-.016	.176	-.09	.928
1	Workflow Config.	settings, suggestions	-.255	.087	-2.95	.004**
2	Code Extensions	extension, chat, js	-.034	.136	-.25	.806
3	Community Info	email, info, request	-.064	.196	-.33	.747
4	UI Selection	select, body, area	-.014	.086	-.17	.867
5	General Usage	time, seems, would	-.010	.087	-.12	.909
6	General Q&A	chat, gpt, discussion	-.111	.100	-1.11	.271
7	Auth & Access	token, auth, sign	-.207	.115	-1.80	.076†
8	IDE Versions	version, studio, plugin	.008	.112	.07	.946

Note. DV = VADER Sentiment Score. Controls: Year, Month, Main Post, Length.

Developer Pack. Consequently, there is a potential concern that our results might be driven by students who may have different motivations (e.g., learning vs. production), time constraints, or coding behaviors compared to professional or non-student developers. We therefore examined student status as a moderator. As shown in Table A13, we find statistically insignificant results across all six outcome variables. This indicates that student status does not significantly moderate the treatment effect. The productivity gains associated with CX adoption are evident for both student and non-student users, confirming that our main results are not an artifact of the student population.

An alternative motivation in OSS development relates to its collaborative nature. To examine whether ACAs differentially affect work on collaborative versus solo projects, we conducted two complementary analyses. First, we provide more direct evidence by examining push patterns across different project types (Table A14). We categorize all repositories based on ownership and

Table A13: Heterogeneous Effect on Student / Non-Student Users

	(1) Ln(Commits)	(2) Ln(Repos)	(3) Ln(Work Hours)	(4) Ln(Sessions)	(5) Langs Added	(6) Project Novelty
Pro × PostAnnouncement	0.05 (0.04)	0.02 (0.02)	0.17** (0.08)	0.18** (0.09)	0.05 (0.27)	-0.01 (0.02)
Pro × PostAnnouncement × Is Student	0.10** (0.05)	0.06*** (0.02)	-0.06 (0.08)	-0.12 (0.09)	0.38 (0.38)	0.02 (0.02)
Pro × PostLaunch	0.10** (0.04)	0.05*** (0.02)	0.08 (0.07)	0.11 (0.08)	0.54** (0.24)	-0.02 (0.02)
Pro × PostLaunch × Is Student	0.03 (0.05)	0.03 (0.02)	-0.03 (0.08)	-0.08 (0.08)	-0.07 (0.36)	0.01 (0.02)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	35100	35100	4247	4247	7150	1293
# Users	1350	1350	635	635		367
Adj. R2 / Likelihood	0.41	0.42	0.27	0.23	-1543.55	1632.85

Note. *Is Student* is a binary that measures the user’s student identity at the time we collected the data. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

collaboration. The results reveal that ACA adoption significantly increases push frequency and size to Personal Projects (columns 2 and 5), while showing no significant effects on contributions to Collaborative Projects (columns 3 and 6). This pattern suggests that developers primarily leverage ACAs for their own projects rather than for external collaborative work. As complementary evidence, we examine whether the benefits of ACAs vary by a developer’s overall propensity for collaborative development. We defined OSS Proclivity as the percentage of a user’s repositories that have received at least one external author contribution, serving as a proxy for how deeply a user is integrated into the collaborative OSS ecosystem. Table A15 presents heterogeneous treatment effects by OSS Proclivity. The interaction terms are largely statistically insignificant across most outcomes, suggesting that the observed treatment effects do not meaningfully vary by a developer’s collaborative orientation. Taken together, these analyses indicate that while ACAs primarily enhance activity on personal projects, their benefits appear broadly applicable regardless of whether developers typically work in collaborative or solo contexts.

Table A14: Push Patterns to Personal and Collaborative Projects

	Push Frequency			Push Size		
	(1) All Projects	(2) Personal Projects	(3) Collaborative Projects	(4) All Projects	(5) Personal Projects	(6) Collaborative Projects
Pro $\times$ PostAnnouncement	0.07** (0.03)	0.06** (0.03)	-0.00 (0.02)	-0.09 (0.06)	-0.06 (0.09)	-0.10 (0.09)
Pro $\times$ PostLaunch	0.10*** (0.03)	0.11*** (0.03)	-0.01 (0.02)	0.11*** (0.04)	0.16** (0.07)	0.01 (0.05)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	35100	35100	35100	3242	2620	1005
# Users	1350	1350	1350	504	466	145
Adj. R2	0.38	0.33	0.42	0.26	0.35	0.30

Note. We measured push frequency by counting the number of public push events made to the target user's owned repositories in each specific week (logarithmized). Push size is the logarithmized weekly average of the number of commits in each push. This information is available in each GitHub user's GH Archive activity records. *All Projects* includes every repository owned by the user. *Personal Projects* denotes repositories owned by the user that have not received any external pull request. *Collaborative Projects* covers repositories owned by the user that has received external pull request.

Table A15: Heterogeneous Effect by User OSS Proclivity

	(1) Ln(Commits)	(2) Ln(Repos)	(3) Ln(Work Hours)	(4) Ln(Sessions)	(5) Langs Added	(6) Project Novelty
Pro $\times$ PostAnnouncement	0.10*** (0.04)	0.05*** (0.02)	0.21*** (0.07)	0.19** (0.09)	0.35 (0.25)	0.00 (0.02)
Pro $\times$ PostAnnouncement $\times$ OSS Proclivity	-0.18 (0.14)	-0.09 (0.06)	-0.56** (0.28)	-0.56 (0.34)	-1.61 (1.14)	-0.01 (0.13)
Pro $\times$ PostLaunch	0.12*** (0.04)	0.07*** (0.02)	0.10 (0.07)	0.10 (0.07)	0.44* (0.24)	-0.02 (0.02)
Pro $\times$ PostLaunch $\times$ OSS Proclivity	-0.10 (0.13)	-0.07 (0.06)	-0.26 (0.35)	-0.22 (0.37)	0.51 (0.70)	-0.02 (0.09)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
User FE	Yes	Yes	Yes	Yes	Yes	Yes
Week FE	Yes	Yes	Yes	Yes	Yes	Yes
N	35100	35100	4247	4247	7150	1293
# Users	1350	1350	635	635		367
Adj. R2 / Likelihood	0.41	0.42	0.27	0.23	-1541.95	0.42

Note. OSS Proclivity is measured as the percentage of repositories that have received at least one pull request, ranging from 0 to 1. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Appendix 9 Other Robustness Analyses

Below, are some supplementary analyses to support the main interpretations we made in the main body. First, we examine whether the increase in developers’ commit volume originates from short-term growth in new project creation with only 1-2 commits for initialization, rather than the persistent engagement incentivized by ACAs’ productivity gains. Since new project launches rarely occur on a weekly basis, we construct a monthly panel and conduct similar DiD analysis with 3 pre- and 5 post-treatment periods using a Poisson model. Result in [A16](#) highlights a transient significant rise in new project creation during the announcement stage, coinciding with a rise in public interest in Gen-AI projects. However, the effect becomes insignificant at a longer time horizon. Combined with the persistent rise in commit volume and worked-on repositories, these findings support that ACAs primarily lead to persistent development activity enhancement, rather than one-off project initiations.

Table A16: GitHub Users’ Monthly Repository Creations (Poisson Count Model)

	(1) Poisson(New Repos)
Pro $\times$ PostAnnouncement	0.26** (0.13)
Pro $\times$ PostLaunch	0.16 (0.12)
AccountAge, Ln(months)	-0.75 (0.88)
User FE	Yes
Month FE	Yes
N	6512
# Users	814
Log Likelihood	-3792.89

Note. We conducted a Poisson regression analysis at the monthly level because new project launches occur infrequently at the weekly level, making it difficult to draw meaningful statistical conclusions. The observation window aligns with our previous weekly-level analyses. Standard errors are clustered at the user level. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Second, we conducted a subgroup analysis on activities within repositories that were unrelated to generative AI to eliminate the influence of the concurrent surge in generative AI-related projects. In [Table A17](#), we used a list of generative AI-related keywords (i.e., generative ai, chatgpt, gpt, openai, anthropic) to classify projects based on whether any of them appeared in the available text information of repositories, such as README documentation, short descriptions, and tags. Our findings revealed a consistent and increasing trend in commit activities when using all the self-owned

repositories. This eliminates the alternative explanation that the rise in development activities is primarily due to Pro users being particularly interested in generative-AI-related development during that specific period.

Table A17: Non-GenAI Project Activities

DV: Ln(Commits)	(1) Non-GenAI Projects
Pro $\times$ PostAnnouncement	0.10*** (0.03)
Pro $\times$ PostLaunch	0.11*** (0.04)
Controls	Yes
User FE	Yes
Week FE	Yes
N	35087
# Users	1350
Adj. R2	0.40

Note. We reported similar estimated coefficients using the same regression equation for column 2 of Table 3. Instead of measuring the treatment effects on development activities across all the repositories owned by the user, we restricted the scope to users' repositories whose descriptions and documentation include keywords related to generative AI. The control variables used are the same as those in our main specifications. Standard errors are clustered at the user level.

Lastly, we included the results of some alternative model as robustness tests. As shown in Table A18, we found a consistent rise in the number of new programming language adopted in treated users' development at the post-launch stage.

Table A18: Poisson Count Model ($\text{Log}(n+1)$)

	(1) Log(Langs Added)
Pro $\times$ PostAnnouncement	0.00 (0.01)
Pro $\times$ PostLaunch	0.01* (0.00)
Controls	Yes
User FE	Yes
Week FE	Yes
N	35100
# Users	1350
Adj. R2	0.01

Note. We implemented $\log(\text{count}+1)$ transformation on new programming language adopted in each week as a robustness check for our Poisson count model that dropped more than half of users in our original matched samples. Standard errors are clustered at the user level.

Appendix 10 Additional LinkedIn Analysis

We report additional robustness checks for our career outcome results using other model specifications in Tables A19 and A20. To summarize, we found consistent results for internal transition outcomes across the two periods in both models. For job promotions, the significant positive association between Pro label and career progression only holds within 6 months in the Logit model. While the coefficient in the LPM model is insignificant, Columns 3 and 6 suggest that the effect direction aligns with what we saw in our main analysis.

Table A19: Career Outcomes (Alternative: Logit)

	6 months			12 months		
	(1) Job Hopping	(2) Internal Transition	(3) Promotion	(4) Job Hopping	(5) Internal Transition	(6) Promotion
treat	-0.03 (0.28)	0.85* (0.44)	0.78* (0.47)	0.07 (0.24)	0.78** (0.33)	0.50 (0.35)
Managerial Role	-0.87** (0.40)	-1.39* (0.76)	0.00 (.)	-0.45 (0.31)	-0.78 (0.47)	-1.31** (0.59)
Employment Type	Yes	Yes	Yes	Yes	Yes	Yes
Education	Yes	Yes	Yes	Yes	Yes	Yes
Industry	Yes	Yes	Yes	Yes	Yes	Yes
Company Size	Yes	Yes	Yes	Yes	Yes	Yes
N	445	373	288	460	402	392
Chi2	24.59	33.17	21.53	34.50	45.59	35.38
Pseudo R2	0.07	0.16	0.16	0.09	0.15	0.15

Note. This table presents the impact of treatment on users' career transitions within 6 and 12 months, using a logit model. The dependent binary outcome variables represent various career movements. Robust standard errors are provided in parentheses. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Additionally, we conduct a mediation analysis to see if the positive association we saw in our cross-sectional results is likely correlated with the rise in users' OSS commit activity. In Tables A21 and A22, we present regression results to test the indirect effect and direct effect on post-treatment career outcomes. Specifically, we control for users' pre-treatment commit level as the baseline instead of using pre-post difference as our mediator to avoid double measurement error and adjust for baseline confounding effects (VanderWeele, 2015).

Our findings in Tables A21 and A22 suggest that the rise in activity observed after CX introduction is unlikely to be the mechanism driving users' internal position changes. Specifically, we find no evidence that commit volume mediates promotion outcomes. Although the treatment coefficient decreases marginally between Columns 2 and 3 in Table A21, the effect of the mediator on the outcome is statistically insignificant, indicating that increased output volume does not predict promotions. Additionally, we observe that the impact of the treatment on the mediator (commit volume) fades and becomes insignificant over the longer (12-month) time span. Collectively, these results imply that raw commit volume is not the channel through which CX access improves career

Table A20: Career Outcomes (Alternative: LPM)

	6 months			12 months		
	(1) Job Hopping	(2) Internal Transition	(3) Promotion	(4) Job Hopping	(5) Internal Transition	(6) Promotion
treat	-0.00 (0.04)	0.04* (0.02)	0.03 (0.02)	0.01 (0.04)	0.06** (0.03)	0.04 (0.03)
Managerial Role	-0.10** (0.04)	-0.05** (0.02)	-0.06*** (0.01)	-0.07 (0.05)	-0.06* (0.03)	-0.07*** (0.03)
Employment Type	Yes	Yes	Yes	Yes	Yes	Yes
Education	Yes	Yes	Yes	Yes	Yes	Yes
Industry	Yes	Yes	Yes	Yes	Yes	Yes
Company Size	Yes	Yes	Yes	Yes	Yes	Yes
N	471	471	471	471	471	471
Adjusted R2	0.03	0.01	0.01	0.05	0.06	0.04

Note. This table presents the impact of treatment on users' career transitions within 6 and 12 months using linear probability model via 'reghdfe' command. The dependent binary outcome variables represent various career movements. Standard errors are clustered at the user level.

outcomes. Instead, the positive association is likely driven by direct effects, suggesting that AI usage may act as a quality signal to employers or drives efficiency gains not captured by simple commit counts.

Lastly, we conducted heterogeneity analyses accordingly to understand the nuance in how ACAs adoption associates with career transition outcomes. As shown in Table A23, we found no significant difference in this association between student and non-student users (Panel A). However, we observed heterogeneous associations for developers with varying employment contracts (Panel B) and work experience (Panel C) over a 12-month horizon: 1) ACA usage among interns is associated with significantly greater internal progression opportunities (internal changes and promotions) compared to other contract types (columns 5-6). 2) Senior employees with more prior work experience appear less likely to leverage ACA usage for external job hopping (column 4), as indicated by the negative interaction term.

Table A21: Mediation Analysis: Commit Intensity and Promotion Outcome

DV: Promotion	In 6 Months			In 12 Months		
	(1) Mediator	(2) Promotion	(3) Promotion	(4) Mediator	(5) Promotion	(6) Promotion
treat	0.110* (0.059)	0.451* (0.273)	0.429 (0.274)	0.074 (0.054)	0.275 (0.191)	0.275 (0.192)
Ln(Commits), Pre	0.676*** (0.034)	0.306** (0.141)	0.199 (0.203)	0.587*** (0.031)	0.081 (0.108)	0.085 (0.145)
Ln(Commits), Post			0.147 (0.206)			-0.007 (0.169)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
N	436	436	436	436	436	436
Adj. R ² / Pseudo R ²	0.503	0.246	0.249	0.472	0.120	0.120
F-statistic / LR χ^2	32.45	35.29	35.75	28.74	30.31	30.31

Note. Columns (1) and (4) report the results of linear regression for the mediator model ($M \sim T + X$). Columns (2), (3), (5), and (6) report the results of probit models for the outcome models ($Y \sim T + X$) and $Y \sim T + M + X$). All models include the pre-treatment activity level (Ln(Commits), Pre). Controls include employment type, education, industry, company size, and manager status. The sample size is held constant across all three models within each mediation analysis. This consistent observation set is required by the mediation package for valid causal inference and is achieved through complete case list-wise deletion on the full set of variables. Categorical variables were processed to pool groups with fewer than 20 observations into an ‘Other’ category. Standard errors are reported in parentheses below the coefficient estimates. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table A22: Mediation Analysis: Commit Intensity and Internal Transition Outcome

DV: Internal Transition	In 6 Months			In 12 Months		
	(1) Mediator	(2) Internal Transition	(3) Internal Transition	(4) Mediator	(5) Internal Transition	(6) Internal Transition
treat	0.110* (0.059)	0.458* (0.239)	0.450* (0.240)	0.074 (0.054)	0.388** (0.182)	0.394** (0.183)
Ln(Commits), Pre	0.676*** (0.034)	0.226* (0.123)	0.180 (0.174)	0.587*** (0.031)	0.087 (0.102)	0.120 (0.137)
Ln(Commits), Post			0.065 (0.179)			-0.057 (0.161)
Controls	Yes	Yes	Yes	Yes	Yes	Yes
Num. Obs.	436	436	436	436	436	436
Adj. R ² / Pseudo R ²	0.503	0.177	0.178	0.472	0.135	0.135
F-statistic / LR χ^2	32.45	30.91	31.03	28.74	39.64	39.76

Note. Columns (1) and (4) report the results of linear regression for the mediator model ($M \sim T + X$). Columns (2), (3), (5), and (6) report the results of probit models for the outcome models ($Y \sim T + X$) and $Y \sim T + M + X$. All models include the pre-treatment activity level (Ln(Commits), Pre). Controls include employment type, education, industry, company size, and manager status. The sample size is held constant across all three models within each mediation analysis. This consistent observation set is required by the mediation package for valid causal inference and is achieved through complete case list-wise deletion on the full set of variables. Categorical variables were processed to pool groups with fewer than 20 observations into an ‘Other’ category. Standard errors are reported in parentheses below the coefficient estimates. * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$

Table A23: Career Outcome Analysis: Heterogeneity

	In 6 Months			In 12 Months		
	(1) Job Hopping	(2) Internal Changes	(3) Promotion	(4) Job Hopping	(5) Internal Changes	(6) Promotion
Panel A. Student vs. Non-student (Linkmod: Is Student)						
Pro	1.43** (0.67)	0.53** (0.24)	0.54** (0.27)	0.03 (0.17)	0.34 (0.21)	0.25 (0.22)
Linkmod	-0.07 (0.22)	-0.07 (0.40)	0.00 (0.41)	0.41* (0.21)	0.02 (0.29)	0.06 (0.29)
Pro × Linkmod	-0.46 (0.34)	-0.08 (0.55)	-0.25 (0.59)	-0.28 (0.32)	0.40 (0.42)	0.15 (0.44)
Managerial Role	-0.49 (0.34)	-0.68** (0.31)	0.00 (.)	-0.25 (0.19)	-0.31 (0.24)	-0.56* (0.29)
Employment Type	Yes	Yes	Yes	Yes	Yes	Yes
Education	Yes	Yes	Yes	Yes	Yes	Yes
Industry	Yes	Yes	Yes	Yes	Yes	Yes
Company Size	Yes	Yes	Yes	Yes	Yes	Yes
N	345	345	266	425	374	364
Chi2	39.20	33.12	21.86	39.43	46.78	33.94
Pseudo R2	0.19	0.16	0.15	0.10	0.15	0.13
Panel B. Intern vs. Other Employment Type (Linkmod: Is Intern)						
Pro	-0.06 (0.16)	0.44** (0.20)	0.43* (0.23)	0.01 (0.14)	0.36** (0.17)	0.23 (0.18)
Linkmod	-0.07 (0.69)	0.00 (.)	0.00 (.)	1.27* (0.71)	-3.59*** (0.30)	-3.74*** (0.27)
Pro × Linkmod	-0.08 (0.91)	0.00 (.)	0.00 (.)	-0.80 (0.85)	3.67*** (0.71)	3.87*** (0.69)
Managerial Role	-0.39* (0.20)	-0.56* (0.30)	0.00 (.)	-0.28 (0.18)	-0.28 (0.21)	-0.50** (0.25)
Employment Type	No	No	No	No	No	No
Education	Yes	Yes	Yes	Yes	Yes	Yes
Industry	Yes	Yes	Yes	Yes	Yes	Yes
Company Size	Yes	Yes	Yes	Yes	Yes	Yes
N	412	368	281	425	397	387
Chi2	11.86	22.98	11.89	22.94	233.82	330.89
Pseudo R2	0.04	0.07	0.06	0.06	0.06	0.06
Panel C. Prior Job Experience (Linkmod: Ln(Prior Work Exp), in months)						
Pro	-0.24 (0.40)	1.43** (0.67)	0.92 (0.68)	0.71* (0.38)	0.83 (0.50)	0.33 (0.51)
Linkmod	-0.22* (0.12)	-0.07 (0.22)	-0.35 (0.25)	-0.09 (0.12)	-0.15 (0.16)	-0.25 (0.16)
Pro × Linkmod	0.08 (0.19)	-0.46 (0.34)	-0.20 (0.38)	-0.40** (0.20)	-0.19 (0.24)	-0.01 (0.25)
Managerial Role	-0.31 (0.22)	-0.49 (0.34)	0.00 (.)	-0.19 (0.19)	-0.22 (0.25)	-0.43 (0.30)
Employment Type	Yes	Yes	Yes	Yes	Yes	Yes
Education	Yes	Yes	Yes	Yes	Yes	Yes
Industry	Yes	Yes	Yes	Yes	Yes	Yes
Company Size	Yes	Yes	Yes	Yes	Yes	Yes
N	412	345	266	425	374	364
Chi2	30.75	39.20	31.53	43.54	50.42	37.41
Pseudo R2	0.09	0.19	0.19	0.11	0.16	0.15

Note. We employed different moderators to assess the heterogeneity across users with varying employment status, type, and work experience. In each heterogeneous analysis group, we used a specific moderator, referred to as *Linkmod*. When using *Is Intern* as the moderator, we excluded Employment Type dummies from the co-variables.

Appendix References

- Baird, M., Carpanelli, M., Xu, B., & Xu, K. (2024). Early evidence on the impact of GitHub copilot on labor market outcomes for software engineers. *LinkedIn Economic Graph Working Paper*, (5).
- Costello, F. J., & Lee, K. C. (2022). Exploring investors' expectancies and its impact on project funding success likelihood in crowdfunding by using text analytics and bayesian networks. *Decision Support Systems*, 154, 113695.
- Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2024). The productivity effects of generative AI: Evidence from a field experiment with GitHub copilot. *An MIT Exploration of Generative AI*.
- Dembovsky, C. (2023). *Responsible ai pair programming with github copilot*. Retrieved February 28, 2023, from <https://github.blog/ai-and-ml/github-copilot/responsible-ai-pair-programming-with-github-copilot/>
- Hoffmann, M., Boysel, S., Nagle, F., Peng, S., & Xu, K. (2024). Generative AI and the nature of work. *Available at SSRN 5007084*.
- Kalliamvakou, E. (2022). Research: Quantifying github copilot's impact on developer productivity and happiness. *The GitHub Blog*.
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub copilot. *arXiv preprint arXiv:2302.06590*.
- Song, F., Agarwal, A., & Wen, W. (2023). The impact of generative ai on collaborative open-source software development: Evidence from github copilot. *Available at SSRN 4856935*.
- VanderWeele, T. J. (2015). *Explanation in causal inference: Methods for mediation and interaction*. Oxford University Press.
- Xu, F., Medappa, P. K., Tunc, M. M., Vroegindeweij, M., & Fransoo, J. C. (2025). Ai-assisted programming may decrease the productivity of experienced developers by increasing maintenance burden. *Available at SSRN 5521379*.
- Yeverechyahu, D., Mayya, R., & Oestreicher-Singer, G. (2024). The impact of large language models on open-source innovation: Evidence from github copilot. *arXiv preprint arXiv:2409.08379*.